Can Reinforcement Learning effectively detect money laundering, and how does it compare to leading supervised learning methods on synthetic transaction data?

Kaan Gogcay
Hogeschool van Amsterdam

June 15, 2026

Abstract

Money laundering causes an estimated €16 billion in losses annually in the Netherlands alone, and financial institutions are legally required under the Wwft to detect and report suspicious transactions. While supervised machine learning, particularly XGBoost, dominates current anti-money laundering (AML) practice, Reinforcement Learning (RL) remains largely unexplored for this domain. This research investigates to what extent RL can effectively detect money laundering and how it compares to leading supervised learning methods on synthetic transaction data. A Deep Q-Network (DQN) was trained and evaluated against an XGBoost baseline on the SAML-D dataset, using Area Under the Precision-Recall Curve (AUPRC) as the primary metric, consistent with industry practice at ING and ABN AMRO. Through iterative feature engineering, XGBoost achieved a test AUPRC of 0.9275, while the best DQN configuration reached only 0.6608. SHAP-based explainability analysis revealed that the DQN failed to exploit the most informative engineered features, producing scores indistinguishable from random guessing for many transactions. The results indicate that RL, in its current form, cannot match supervised methods for AML transaction monitoring on tabular data.

1 Introduction

This chapter introduces the problem of money laundering, reviews the current state of the art, and identifies the research gap that motivates this work.

1.1 Context Analysis

To understand the problem this research addresses, this section introduces money laundering, its detection, and the limitations of traditional approaches.

1.1.1 Money Laundering

Money laundering is the process of concealing the illegal origin of funds to make them appear legitimate, allowing criminals to use the proceeds in the legal economy FIU-the Netherlands (2023); Financial Crimes Enforcement Network (FinCEN) (nd). The scale of this problem is significant: The Netherlands alone sees an estimated 16 billion euros laundered annually, ranking eighth globally in terms of laundered funds Menon and Nguyen (2026). To combat this, financial institutions in the Netherlands are legally required to monitor and report suspicious transactions under the *Wet ter voorkoming van witwassen en financieren van terrorisme (Wwft)* Ministerie van Financiën (2008). These funds often come from crimes such as drug trafficking, fraud, illegal pornography, tax evasion, robbery, blackmail, bribery, piracy, people trafficking and many more malicious activities FIU-the Netherlands (2023); Cox (2012); Menon and Nguyen (2026). Once obtained, the money cannot be spent freely, so criminals must disguise its illegal origin. This is achieved through the money-laundering process, which typically occurs in three stages: placement, layering and integration Cox (2012).

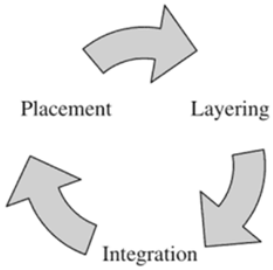

Figure 1: The money-laundering process

Placement Placement is the first step in the laundering process. In this step, criminals attempt to introduce their illegal funds into the financial system. This is often associated with depositing cash into bank accounts, but placement is not limited to plain deposits. Another popular example of introducing criminal money to the financial system is to buy physical assets, such as paintings, antiques and boats Cox (2012).

Layering Once criminally earned funds have been introduced to the financial system, criminals try to hide the origin of this money. This can be done in various ways, such as investing in legitimate assets, buying and selling collectibles, or making multiple smaller transactions across different accounts. The more these funds are moved back and forth, the more layers are formed and the harder it becomes to trace their original source. Cox (2012).

Integration Once enough layering has taken place and the origin of the funds has become sufficiently difficult to trace, criminals can freely integrate their laundered money into the legitimate economy. This can be done by investing in businesses, purchasing property, buying jewellery, or other assets that appear legitimate Cox (2012).

1.1.2 Money Laundering Detection

Money laundering detection or AML (anti-money laundering) refers to the process of detecting money laundering. AML technology has changed significantly over the past years, gradually shifting from rule-based systems towards semi-automated machine learning workflows

1.1.3 Traditional AML Methods and its Limitations

Rule-based systems, the traditional approach to AML, became one of the first forms of automated detection and are still used in current day Attard and de Bruijn (2026); Menon and Nguyen (2026); Dobbelstein and Yeng (2026). These systems typically monitor transactions and generate alerts for activity exceeding thresholds, such as amounts over $10,000, amounts just below $10,000, or multiple smaller transactions within 24 hours that collectively surpassed $10,000 Weber et al. (2018). Over time, banks have expanded the amount of rules in their systems in an attempt to catch as many suspicious transactions as possible. ING, a major bank in the Netherlands, reported using close to 500 rules as of March 2026 Attard and de Bruijn (2026). DLL, [CLASSIFIED] Dobbelstein and Yeng (2026), illustrating that rule-based systems remain in active use across institutions of varying scale. While rule-based systems improved efficiency over manual review, their limitations in handling complex laundering patterns and large-scale data motivated machine learning as an additional layer to these systems Menon and Nguyen (2026).

1.2 Literature Overview

1.2.1 State of the Art

This section provides an overview of the current state-of-the-art regarding Machine Learning applications within Anti-Money Laundering. The field is currently dominated by supervised learning methods, particularly tree-based ensemble approaches, as evidenced by both academic literature and industry benchmarks.

IEEE-CIS Fraud Detection Competition In 2019, Kaggle hosted the *IEEE-CIS Fraud Detection Competition* in collaboration with Vesta Corporation, an event that remains a foundational milestone for machine learning in the financial sector IEEE Computational Intelligence Society (2019). By providing a massive, real-world dataset directly from Vesta's e-commerce transactions, the competition set a definitive benchmark for the state-of-the-art in transaction monitoring and fraud detection algorithms.

Vesta, a leader in e-commerce payment solutions, shared this data to challenge researchers to improve fraudulent transaction detection. The competition aimed to increase the detection of criminal activity while reducing false positives.

The competition allowed researchers to join individually or in teams, with the flexibility to submit an unlimited number of solutions to refine their results. The scale of the competition is reflected in the final statistics: 7,389 individual researchers, 6,351 unique teams, 125,219 submissions and a $20,000 prize pool.

Of the five top performing teams, three publicly shared their solutions. An analysis of these reveals a heavy reliance on supervised learning, specifically, Gradient Boosting Decision Trees (GBDT):

- **1st Place (Score: 0.9677)**: The winning solution utilized a stacked ensemble of CatBoost, XGBoost, and LightGBM. The team also developed a Neural Network, but it was dropped from the final submission because it underperformed compared to the Gradient Boosting models Deotte and Yakovlev (2019).
- **2nd Place (Score: 0.9672)**: This team achieved their ranking using a blend of LightGBM, CatBoost, and XGBoost. Their performance was further boosted by a temporal post-processing step that overrode predictions based on a user's past fraud history Bryansky et al. (2019).
- **5th Place (Score: 0.9424)**: The 5th place solution relied on a simple average of four independent LightGBM models. They also created a specialized model focused on "single transaction" users, the output of which was used as a feature for their main LightGBM ensemble Team Lions (2019).

Recent developements within Supervised Learning A recent survey (2017–2023) analysed papers that cover supervised AML models trained on real-world financial data, summarizing results across studies into a comparative overview Youssef et al. (2023).

As shown in [Table 1, Youssef et al. (2023)], the most frequent used supervised methods in this time period are Logistic Regression (6 studies), Random Forest (5), XGBoost (4), Decision Tree (3) and Support Vector Machines (3). The table also indicates what models have been rarely explored recently (only once) on real banking data, also including non-supervised methods, namely: Artificial Neural Networks, Gru, LSTM, Transformers, Decision Regression Tree, Naive Bayes, K-Nearest Neighbors, Feedforward Network, AdaBoost, Graph Deep Learning and Graph Convolutional Network. It is worth noting that the last three of these rarer methods were only applied to cryptocurrency transaction datasets rather than traditional banking data, leaving their potential on traditional banking data unexplored.

1.2.2 Reinforcement Learning, an Emerging Method

To date, Reinforcement Learning has barely been explored within AML. However, some research exists on the related topic of fraud detection, including:

One paper proposes FraudGNN-RL Cui et al. (2025), which combines GNNs with RL for adaptive fraud detection. Transactions are modeled as a dynamic graph, and a Deep Q-Network dynamically adjusts the detection threshold to adapt to evolving fraud patterns. Experiments across three real-world datasets reportedly achieved AUPRC scores of 0.647, 0.652 and 0.649 outperforming baselines which include XGBoost, Isolation Forest, and several other methods.

Another paper Qayoom et al. (2024) also explores RL utilizing a Deep Q-Network, but without the use of graph-based methods. Rather than scoring individual transactions directly, the agent observes a rolling window of the last 30 transactions per cardholder and learns to flag deviations from personal spending behaviour. The model was trained and tested on a publicly available Kaggle credit card fraud dataset, achieving 97.1% validation accuracy. It is worth noting that this paper focuses on credit card fraud rather than money laundering specifically, though both problems are fundamentally based on transaction monitoring.

1.3 Industry Insights

To ground this research in real-world practice, two sources of industry knowledge were consulted. First, presentations were attended from data scientists at two major Dutch banks: ING Attard and de Bruijn (2026) and ABN AMRO Menon and Nguyen (2026). Second, an interview was conducted at De Lage Landen (DLL), a financial services and leasing company based in the Netherlands, as part of the field research component of this study Dobbelstein and Yeng (2026).

1.3.1 ING

ING uses a workflow including a rule-based system and a human-in-the-loop to generate labels for the supervised models Attard and de Bruijn (2026). ING currently operates approximately 500 static rules to flag potentially suspicious transactions. When a rule fires, a human investigator reviews the transaction and determines whether it is genuinely suspicious. This judgment then becomes the label. This labeling process is therefore inherently dependent on both the quality of the existing rules and human judgment.

ING exclusively uses supervised classification models for transaction monitoring. Tree-based models, particularly XGBoost, being the preferred approach due to their strong performance and explainability. The data scientists would like to explore more complex models such as neural networks and graph-based approaches but face practical barriers, including engineering constraints given the scale of the data (approximately 8 billion transactions) and the difficulty of obtaining regulatory sign-off for less interpretable models.

Given that potential financial crime represents only 1–2% of all transactions, the data is highly imbalanced. ING therefore uses the Area Under the Precision-Recall Curve (AUPRC) as their primary evaluation metric, a measure well suited to imbalanced data, targeting a minimum recall of 0.95.

1.3.2 ABN AMRO

ABN AMRO takes a broader approach than ING, combining both supervised and unsupervised models within their transaction monitoring pipeline Menon and Nguyen (2026). Each transaction passes through two parallel flows: one using supervised models, including XGBoost, decision trees, and random forests, to detect known money laundering patterns, and another using unsupervised models, including auto-encoders, clustering, and isolation forest, to detect unknown or anomalous behaviour. Both flows feed into manual review by investigators before anything is reported to the FIU.

Similar to ING, labels at ABN AMRO originate from rule-based systems, a process they acknowledge is error-prone. They expressed a desire to move towards ML-based labelling in the future.

A notable practice at ABN AMRO is their operational use of SHAP values. Every alert generated by the model is accompanied by a ranked list of the top three features that contributed to the alert, along with a non-technical explanation for investigators.

Similar to ING, ABN AMRO uses AUPRC as their primary evaluation metric. The detection threshold is not fixed by the data scientists alone. It is determined in consultation with business stakeholders, balancing the desire to catch as many cases as possible against the number of alerts investigators can realistically review.

1.3.3 DLL

[CLASSIFIED]

1.4 Identified Gap

Despite the promise of Reinforcement Learning in adaptive fraud detection, its application to AML on tabular transaction data remains largely unexplored. The literature reveals several concrete gaps.

First, graph-based methods such as Graph Convolutional Networks and Graph Deep Learning, which are well suited to the relational nature of banking transactions, have only been applied to cryptocurrency datasets Alarab et al. (2020); Weber et al. (2019). Their potential on traditional banking transaction data remains unexplored.

Second, while Reinforcement Learning has shown promise in fraud-specific transaction monitoring, existing RL approaches have not been applied to AML transaction monitoring, where the goal is to detect a wide range of laundering patterns rather than detecting only fraud. FraudGNN-RL combines GNN with RL but was evaluated on a general fraud dataset rather than a dedicated AML dataset Cui et al. (2025). The only pure RL approach found in the literature targets credit card fraud, which, while related, is a fundamentally different problem from money laundering Qayoom et al. (2024).

1.5 Scope

Both RL on tabular data and RL combined with graph-based methods represent unexplored directions in the AML literature, making either a valid research choice. This research focuses on the tabular direction, as financial institutions confirmed using tabular transaction data in their AML pipelines Attard and de Bruijn (2026). Graph-based approaches are therefore out of scope.

Since the context of this research involves banking institutions as stakeholders, it may be assumed that real banking data is used. This is not the case. Real-world AML datasets are not publicly available due to privacy and legal constraints, making synthetic datasets the only viable option. Cryptocurrency transaction datasets are equally out of scope, as they do not reflect the traditional banking context this research addresses.

Furthermore, while the RL papers discussed in the literature review originate from the broader fraud detection domain, this research is scoped specifically to money laundering detection. Credit card fraud and other forms of financial fraud are related but distinct problems and are not the subject of this research.

Finally, given that the Wwft requires financial institutions to be able to explain AML decisions Ministerie van Financiën (2008), explainability is addressed in this research. The explainability analysis serves to assess whether flagged transactions can be interpreted in a meaningful way by investigators, rather than providing a legal judgment on model deployability.

1.6 Research Question

Within the defined scope, this research investigates the following main question:

> *To what extent can Reinforcement Learning effectively detect money laundering, and how does it compare to leading supervised learning methods on transaction data?*

To answer this, the following subquestions are addressed:

1. How can Reinforcement Learning be applied to AML transaction monitoring?
2. How does Reinforcement Learning perform compared to leading supervised learning methods?
3. What are the benefits and limitations of Reinforcement Learning for AML?

2 Background

This chapter provides the theoretical and ethical foundation for this research, building on the identified gap by covering the techniques used, a comparison of related work, and the considerations that shaped the research design.

2.1 Relevant Literature on Techniques

2.1.1 Reinforcement Learning and Deep Q-Networks

Reinforcement Learning (RL) is a machine learning paradigm in which an agent learns to make decisions by interacting with an environment. Rather than learning from a labeled dataset, the agent receives a reward or penalty based on its actions and uses this feedback to improve its policy over time. This makes RL particularly suitable for sequential decision-making problems where the optimal action depends on context and may change over time. A widely used RL algorithm is the Deep Q-Network (DQN), which combines Q-learning with a neural network to approximate the value of taking a given action in a given state.

2.1.2 Explainability with SHAP

Explainability is a legal expectation in AML under the Wwft Ministerie van Financiën (2008), and this research therefore uses SHAP (SHapley Additive exPlanations) to interpret model decisions. SHAP is an explainability method that can be applied to any machine learning model, assigning each input feature a contribution score for a given prediction, making it particularly suitable for interpreting black-box models such as the Deep Q-Network used in this research. In the context of AML, SHAP has already been applied in Tertychnyi et al. (2022) (also discussed in the survey by Youssef et al. (2023)) and by ABN AMRO Menon and Nguyen (2026) to identify which features contributed most to a model flagging a customer as high-risk.

2.2 Related Work Comparison

As established in the literature overview, only two papers were found that apply Reinforcement Learning to transaction monitoring. Table 1 provides a structured comparison between these papers and the research proposed in this thesis.

Table 1: Comparison of related work

	Cui et al. Cui et al. (2025)	**Qayoom et al. Qayoom et al. (2024)**	**This research**
Method	GNN + RL	RL	RL
Domain	General fraud	Credit card fraud	AML
Data	PaySim, Credit Card 2023, IEEE-CIS	ULB Credit Card Fraud	SAML-D
Metrics	AUC-ROC, AUPRC, F1, Recall@k%	Accuracy	AUPRC
Explainability	-	-	SHAP

The comparison highlights that while both related papers apply Reinforcement Learning to transaction monitoring, neither targets money laundering specifically. FraudGNN-RL relies on graph structure, which are out of scope for this research, while Qayoom et al. focus on credit card fraud, which is a fundamentally different problem from money laundering.

A notable methodological concern with Qayoom et al. is their exclusive use of accuracy as evaluation metric. Given that their dataset is highly imbalanced (with only 0.172% fraudulent transactions) accuracy alone is a misleading indicator of model performance. A model that classifies every transaction as legitimate would still achieve over 99% accuracy. Metrics such as AUC-ROC, AUPRC, precision, recall, and F1-score are better suited for imbalanced fraud detection tasks, as used by FraudGNN-RL. This research therefore adopted AUPRC as its evaluation metric, consistent with industry practice at both ING and ABN AMRO Attard and de Bruijn (2026); Menon and Nguyen (2026).

2.3 AI Suitability Justification

The suitability of AI for AML transaction monitoring is supported by the insights gathered from presentations by data scientists at ING Attard and de Bruijn (2026) and ABN AMRO Menon and Nguyen (2026).

The fundamental limitation of rule-based systems is that they assume the programmer has a complete picture of all criminal patterns. As ABN AMRO noted, laundering patterns are too diverse and dynamic for rules to cover every scenario; at some point, the number of rules simply becomes unmanageable Menon and Nguyen (2026). ING's reported use of approximately 500 static rules proves this problem in practice Attard and de Bruijn (2026). Machine learning addresses this by learning complex patterns directly from data, allowing the system to detect suspicious behaviour without relying on predefined thresholds.

A further argument for AI is the nature of money laundering itself. As ABN AMRO observed, individual transactions may not appear suspicious by itself, it is the patterns across transactions that reveal criminal activity Menon and Nguyen (2026). Machine learning models are well suited to capture these patterns, particularly at the layering stage of the laundering process, which is both the most difficult stage for rules to catch and the most suitable to model-based detection Menon and Nguyen (2026).

2.4 Ethical and Societal Considerations

The application of machine learning to AML transaction monitoring raises several ethical and societal considerations. Beyond these concerns, effective AML detection also carries direct societal benefits by disrupting criminal activity and protecting the integrity of the financial system.

2.4.1 False Positives and Customer Impact

A key ethical concern in AML systems is the consequences of false positives. When a legitimate customer is incorrectly flagged as suspicious, they may face restricted access to banking services, delayed transactions, or reputational harm. Both ING and ABN AMRO explicitly aim to reduce false positives in their systems Attard and de Bruijn (2026); Menon and Nguyen (2026).

2.4.2 Bias and Fairness

Machine learning models trained on historical transaction data risk inheriting biases present in that data. For example, if certain demographic groups or transaction types were historically flagged more often, a model may learn to replicate these patterns. ING acknowledged the existence of dedicated teams for bias and fairness assessment Attard and de Bruijn (2026), reflecting the seriousness of this concern.

2.4.3 Labeling Bias

A subtler but important issue is that the labels used to train supervised AML models are themselves derived from rule-based systems and human investigators. As both banks acknowledged, this process is error-prone Attard and de Bruijn (2026); Menon and Nguyen (2026). A model trained on these labels may therefore learn to replicate the limitations and biases of the existing system rather than detecting true financial crime.

2.4.4 Explainability as an Ethical Requirement

Explainability is not only a legal requirement under the Wwft Ministerie van Financiën (2008) but also an ethical one. When a model flags a customer as suspicious, investigators and potentially customers themselves deserve to understand why. ABN AMRO's operational use of SHAP values to provide investigators with a ranked explanation of each alert reflects this principle in practice Menon and Nguyen (2026).

2.4.5 Data Privacy

The use of personal financial data raises privacy concerns. In the Netherlands, the GDPR (known locally as the AVG) restricts what personal data can be used for model training European Parliament and Council of the European Union (2016). ING noted that sensitive attributes such as name, address, gender, nationality, and country of origin are not accessible for model training Attard and de Bruijn (2026).

3 Methodology

This chapter describes how the research was structured, including the experimental design, the type of study, the dataset, the models, and the evaluation strategy.

3.1 Research Overview

This research follows an experimental design comparing two models on the same AML dataset under identical conditions. XGBoost was selected as the supervised learning baseline and a Deep Q-Network (DQN) as the Reinforcement Learning model. Both models were trained and evaluated on the SAML-D dataset, a synthetic AML dataset consisting of 9.5 million tabular banking transactions across 28 laundering typologies. The primary evaluation metric is the Area Under the Precision-Recall Curve (AUPRC), and SHAP values were applied to both models to satisfy the explainability requirement imposed by the Wwft Ministerie van Financiën (2008). The DQN is considered to outperform the baseline if it achieves a higher AUPRC score under the same experimental conditions.

3.2 Requirements

This research operates within defined legal, functional, and technical requirements. The legal requirements are grounded in the Wwft Ministerie van Financiën (2008) and GDPR European Parliament and Council of the European Union (2016), the functional requirements define what the model must be capable of doing, and the technical requirements define how and under what conditions it must be built. The full list of requirements is provided in Appendix A.

3.3 Data

This section describes the dataset selected for this research, its characteristics, the preprocessing steps applied, and the risks and biases associated with its use.

3.3.1 Dataset Selection

Real-world AML datasets are not publicly available due to privacy and legal constraints Jensen et al. (2023), making synthetic datasets the only viable option. Multiple synthetic datasets were considered, as shown in Table 2. From a representativeness perspective, SynthAML would have been the most suitable dataset, as it is the only one grounded in real EU banking data, synthesized from actual transactions at Spar Nord, a Danish bank operating under the same EU regulatory framework as Dutch banks Jensen et al. (2023). However, SynthAML contains only 20,000 transactions and does not include a data generator, making it insufficient for training deep learning models such as a DQN. The *money laundering data* dataset Mahootiha (2023) was also considered but rejected because it is generated from only three simplified rules covering the placement, layering and integration stages, resulting in insufficient typological diversity for training a model intended to detect a wide range of laundering patterns.

SAML-D was selected because it includes the `Laundering_type` field, which is absent from IBM AML. While IBM AML models 8 laundering patterns, SAML-D covers 28 distinct typologies, labelling not only whether a transaction is suspicious, but also which laundering pattern it belongs to. This allows for a more extensive evaluation of model performance per pattern, which can help optimise the model, and also provides interesting insights during stakeholder evaluation, allowing investigators to understand which patterns the model captures and which it does not. For a detailed overview of the dataset features and laundering typology descriptions, refer to Appendix B and Appendix C respectively.

Table 2: AML Dataset Comparison

Dataset	Rows	Patterns	Features	Type	Labels	% fraud	Data Generator
IBM AML (HI-Large)	180M	8	11	Synthetic	Yes	0.125%	Yes
IBM AML (HI-Medium)	32M	8	11	Synthetic	Yes	0.110%	Yes
IBM AML (HI-Small)	5M	8	11	Synthetic	Yes	0.100%	Yes
IBM AML (LI-Large)	176M	8	11	Synthetic	Yes	0.057%	Yes
IBM AML (LI-Medium)	31M	8	11	Synthetic	Yes	0.050%	Yes
IBM AML (LI-Small)	7M	8	11	Synthetic	Yes	0.051%	Yes
SAML-D	9.5M	28	12	Synthetic	Yes	0.104%	Yes
SynthAML	20,000	-	2	Synthetic	Yes	17.175%	No
money laundering data	2,340	3	7	Synthetic	Yes	59.786%	Yes

3.3.2 Data Split

The dataset was split before any exploration or feature engineering took place, ensuring that all subsequent analysis was performed on training data only and preventing any possibility of data leakage into the modelling phase. Since the dataset is temporal, a chronological split was used rather than a random one. In a real AML system, a model always scores transactions using only past information, so a random split would allow the model to train on future transactions, constituting data leakage. Walk-forward validation was considered as an alternative to preserve the temporal structure across multiple folds, but given the dataset size of 9.5 million transactions, training multiple models across folds would be computationally expensive, especially for the DQN. A fixed chronological 70/15/15 split was used instead, consistent with the experimental setup of Oztas et al. Oztas et al. (2023).

3.3.3 Feature Engineering

Features were engineered through three iterations, each driven by per-pattern detection failures of the XGBoost model. Starting from 12 raw features (validation AUPRC = 0.0770), account-level aggregates such as transaction counts, amount statistics, and fan-ratios were added (26 features, AUPRC = 0.7704), following Harris et al. Harris et al. (2021). Analysis of remaining failures, particularly on `Fan_Out`, showed that suspicious transactions were concentrated in sender-receiver pairs with no

prior history, motivating the addition of `pair_tx_count` (27 features, AUPRC = 0.9087). The remaining lowest-performing patterns (`Scatter-Gather`, `Layered_Fan_In`, `Cycle`) involve money movement across multiple accounts over time, which would require rolling time-window or graph-based features beyond this research's scope. The full iteration history is provided in Appendix D.

3.4 Evaluation Metric

As established in Section 3.3, the SAML-D dataset is highly imbalanced with only 0.104% of transactions labelled as laundering. Under such extreme class imbalance, commonly used metrics such as accuracy become misleading, since a model that classifies every transaction as normal would achieve 99.896% accuracy while detecting zero laundering cases. This motivates the use of a metric specifically designed for imbalanced classification.

This research uses the Area Under the Precision-Recall Curve (AUPRC) as the primary evaluation metric. AUPRC was selected for three reasons. First, it is specifically designed for imbalanced datasets, evaluating model performance exclusively on the minority class across all classification thresholds. Second, both ING and ABN AMRO use AUPRC as their primary evaluation metric in production Attard and de Bruijn (2026); Menon and Nguyen (2026), making it the industry standard for AML transaction monitoring in the Netherlands. Third, it evaluates precision and recall simultaneously across all possible classification thresholds, enabling business stakeholders to select an operating threshold that matches their operational capacity Attard and de Bruijn (2026); Menon and Nguyen (2026). For this reason, this research does not prescribe a fixed operational threshold. The choice of threshold and the corresponding precision-recall trade-off, is left to be determined in production based on investigator capacity.

Throughout this research, AUPRC is used to compare model iterations, with a higher score indicating better overall performance. Each iteration is evaluated on the validation set, and the test set is reserved exclusively for final evaluation of the best-performing configuration.

3.5 Models

This research implements two models and evaluates them under identical conditions. XGBoost serves as the supervised learning baseline and a Deep Q-Network (DQN) serves as the Reinforcement Learning model. Both models are trained on the same dataset using the same evaluation metric, with the goal of determining whether RL can match or outperform the supervised learning standard on AML detection.

3.5.1 Baseline Definition: XGBoost

XGBoost was selected as the supervised learning baseline for three reasons. First, it is the most frequently used method in the AML literature, appearing in four of the studies surveyed by Youssef et al. Youssef et al. (2023). Second, the top-performing teams in the IEEE-CIS Fraud Detection Competition relied heavily on XGBoost as part of their winning solutions Deotte and Yakovlev (2019); Bryansky et al. (2019); Team Lions (2019), establishing it as the state of the art for tabular fraud detection. Third, both ING and ABN AMRO confirmed that XGBoost is their primary model in production Attard and de Bruijn (2026); Menon and Nguyen (2026). Using XGBoost as the baseline therefore ensured that the RL model was benchmarked against a method that is both academically established and practically relevant.

3.5.2 XGBoost Iterations

XGBoost was developed across three iterations. Iteration 1 used default hyperparameters, overriding only `scale_pos_weight` (∼982) to handle class imbalance XGBoost Developers (2024), `eval_metric` (`aucpr`), `early_stopping_rounds` (50), `enable_categorical`, and `random_state` (42), achieving a validation AUPRC of 0.9087.

Iteration 2 applied a grid search over `eta`, `booster`, `subsample`, and `min_child_weight`, the parameters identified as most tunable by Probst et al. Probst et al. (2019), with values drawn from Probst et al.'s reported ranges, the 1st-place IEEE-CIS Fraud Detection solution Deotte and Yakovlev (2019), and XGBoost defaults XGBoost Developers (2024), evaluating all 150 combinations and improving validation AUPRC to 0.9107.

Iteration 3 applied Bayesian optimisation via Optuna over the same parameter space, achieving a validation AUPRC of 0.9136 across 100 trials (Trial 22 best).

As shown in Table 3 and Figures 2-3, improvements were marginal (0.0020 and 0.0029 validation AUPRC respectively), suggesting diminishing returns from tuning alone; further gains would likely require additional feature engineering. Iteration 3 is therefore used as the XGBoost benchmark for comparison against the DQN. Full parameter configurations and search spaces are provided in Tables 27, 28, and 29.

Table 3: XGBoost iteration results

Iteration	Parameters	Val AUPRC	Test AUPRC
Iteration 1	Defaults	0.9087	0.9232
Iteration 2	Grid search	0.9107	0.9260
Iteration 3	Bayesian optimisation	0.9136	0.9275

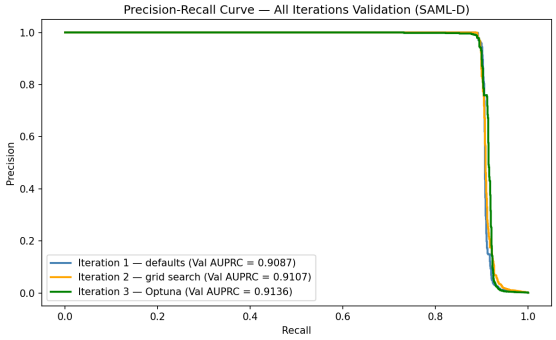

Figure 2: Precision-recall curves for all XGBoost iterations on the validation set

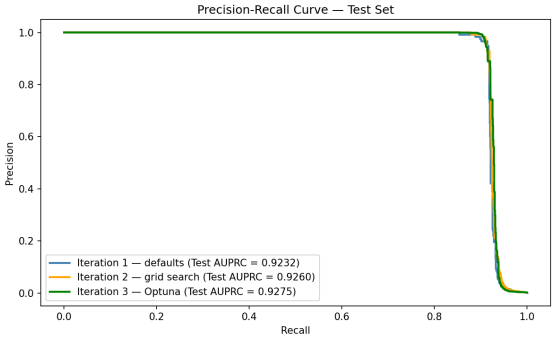

Figure 3: Precision-recall curves for all XGBoost iterations on the test set

3.5.3 Reinforcement Learning: Deep Q-Network

The Deep Q-Network (DQN) was selected as the Reinforcement Learning technique because both RL papers found in the literature apply DQN to transaction monitoring Cui et al. (2025); Qayoom et al. (2024), making it the most documented approach for this domain. The DQN was trained across five iterations, each changing a single aspect of the configuration to isolate the effect of each design choice. Before describing the iterations, data preprocessing, the MDP formulation and network architecture are introduced, as these remained consistent across most iterations.

Data Preprocessing Unlike XGBoost, which handles categorical features natively, neural networks require numeric inputs. The five categorical columns (`payment_currency`, `received_currency`, `sender_bank_location`, `receiver_bank_location`, `payment_type`) were one-hot encoded, expanding the feature set from 27 to 91.

One-hot encoding alone does not address the large differences in feature scale: `Amount` ranges from 3.73 to 12,618,498.40, while `pair_tx_count` ranges from 1 to 62, and one-hot features are bounded between 0 and 1. Since gradient descent is sensitive to input scale, larger-magnitude features would otherwise dominate training.

To address this, standardisation (z-score scaling) was chosen over min-max normalisation due to the heavy-tailed distribution of features such as `Amount` (mean 8,799, max 12,618,498, standard deviation exceeding three times the mean), where normalisation would compress most values into a narrow range near zero, discarding useful variation Han et al. (2011). The scaler was fitted exclusively on the training set and applied to the validation and test sets to prevent data leakage.

MDP Formulation Reinforcement Learning requires framing the problem as a Markov Decision Process (MDP). Following the formulations of Vimal et al. (2021), Lin et al. (2019) and Zhinin-Vera et al. (2020), the AML detection task is adapted as follows.

State. Each transaction is represented as a state, described by the full 91-element feature vector produced after one-hot encoding and standardisation.

Action space. The agent chooses between two discrete actions at each step: action 0 (predict normal) and action 1 (predict suspicious). The Q-network outputs a Q-value for each action, and the agent selects the action with the highest Q-value.

Reward function. The reward function is grounded in the Imbalanced Classification Markov Decision Process (ICMDP) established by Lin et al. (2019). The minority class receives a reward of ± 1.0 and the majority class receives $\pm\lambda$, with the smaller magnitude for the majority class reflecting the greater impact of correctly classifying laundering transactions. Two values of λ are tested in separate iterations. Lin et al. report that the optimal λ equals the class imbalance ratio ρ, giving $\lambda = 0.001039$ for SAML-D. Zhinin-Vera et al. Zhinin-Vera et al. (2020) adopt the same ICMDP reward structure but report $\lambda = 0.1$ as the optimal value on their dataset, which is considerably larger than what Lin et al.'s formula would prescribe for their imbalance ratio of $\rho = 0.001728$. Since both values claim good results on their respective datasets, both are tested to determine which performs better on SAML-D.

Table 4: DQN reward structure

Prediction	Ground truth	Reward	Rationale
Suspicious (1)	Laundering (1)	$+1.0$	True positive: correct minority class detection
Normal (0)	Laundering (1)	-1.0	False negative: highest penalty
Suspicious (1)	Normal (0)	$-\lambda$	False positive: incorrect majority class
Normal (0)	Normal (0)	$+\lambda$	True negative: correct majority class

Episode structure. Because SAML-D is a static tabular dataset rather than a sequential environment, each transaction is treated as an independent single-step episode, following Lin et al. (2019) and Zhinin-Vera et al. (2020), who both process one sample per episode. The discount factor γ is set per iteration following the respective source paper.

Network Architecture The Q-network maps a transaction's 91-element feature vector to a Q-value for each of the two actions. The architecture follows Qayoom et al. Qayoom et al. (2024) directly, whose network is specifically designed for static tabular transaction data.

Table 5: DQN Q-network architecture

Layer	Neurons	Activation
Input	91	–
Hidden 1	124	ReLU
Hidden 2	62	ReLU
Hidden 3	31	Tanh
Hidden 4	15	Linear
Output	2	Linear

The output layer produces two raw Q-values with no activation function, allowing Q-values to be positive or negative to reflect how desirable each action is in a given state. Qayoom et al. use a sigmoid output because their network outputs a single fraud probability. Here the output has two neurons, one per action, requiring no activation.

Training Mechanics Four core training techniques are implemented consistently across all iterations.

Epsilon-greedy exploration. The agent selects a random action with probability ε and the greedy action otherwise. ε starts at 1.0 and decays each step to a minimum of 0.01, following Lin et al. (2019). This ensures the agent explores early in training before increasingly exploiting learned knowledge.

Experience replay. Transitions (state, action, reward, next_state, done) are stored in a fixed-size replay buffer, from which random mini-batches are sampled for training. This breaks the correlation between consecutive transitions and stabilises gradient updates, a core DQN technique introduced by Mnih et al. (2015) and adopted by all three source papers. Across these papers, the buffer is consistently large enough to store the entire training set (e.g., Zhinin-Vera et al. (2020) use 200,000 for 199,365 training transactions), suggesting the intent is to retain the full diversity of training experiences at all times. Applying this principle to SAML-D's 6,653,396 training transactions, a buffer of equivalent size is used.

Target network. A separate target network is maintained as a periodically updated copy of the Q-network, following Mnih et al. (2015) and Qayoom et al. (2024) who introduced it as a core DQN stabilisation technique.

Loss. Mean squared error between the predicted Q-values and the Bellman target, optimised with Adam. Lin et al. (2019) use a learning rate of 0.00025 and Qayoom et al. (2024) use 0.0001. Since Lin et al. operate on a dataset scale more comparable to SAML-D than Qayoom et al., a learning rate of 0.00025 is adopted following Lin et al. (2019).

Iteration 1: Lin et al. reward function ($\lambda = \rho$) Iteration 1 implements the reward function of Lin et al. (2019), who establish that the optimal λ equals the class imbalance ratio ρ, giving $\lambda = 0.001039$ for SAML-D ($\rho = 0.001039$). Batch size was scaled up from 20 to 256 and epsilon decay reduced proportionally, both to match SAML-D's 6,653,396 training transactions without collapsing exploration prematurely or producing an impractically slow training loop. The full configuration, including sources for each parameter, is shown in Table 6. This iteration achieved a validation AUPRC of 0.5994 (Figures 4, 5).

Table 6: DQN Iteration 1 hyperparameters

Parameter	Value	Source
`epochs (max)`	100	upper bound only; early stopping determines actual epoch count
`memory_size`	6,653,396	Lin et al. (2019) principle
`batch_size`	256	scaled up from Qayoom et al. (2024) (20) to match data volume
`gamma` (γ)	0.1	Lin et al. (2019)
`lambda` (λ)	0.001039	Lin et al. (2019): $\lambda = \rho$
`epsilon_start`	1.0	Lin et al. (2019)
`epsilon_decay`	0.00001	scaled down from Qayoom et al. (2024) (0.0003) proportional to dataset size
`epsilon_min`	0.01	Lin et al. (2019)
`lr`	0.00025	Lin et al. (2019)
`train_freq`	10	Qayoom et al. (2024)
`target_update_freq`	20	Qayoom et al. (2024)
`early_stopping_patience`	10	–
`early_stopping_min_delta`	0.001	–

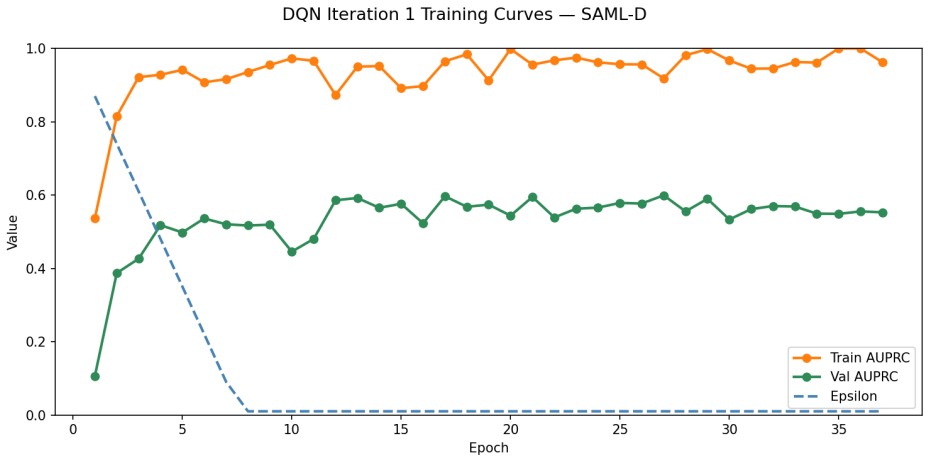

Figure 4: DQN Iteration 1 training curves: train AUPRC, validation AUPRC, and epsilon decay per epoch.

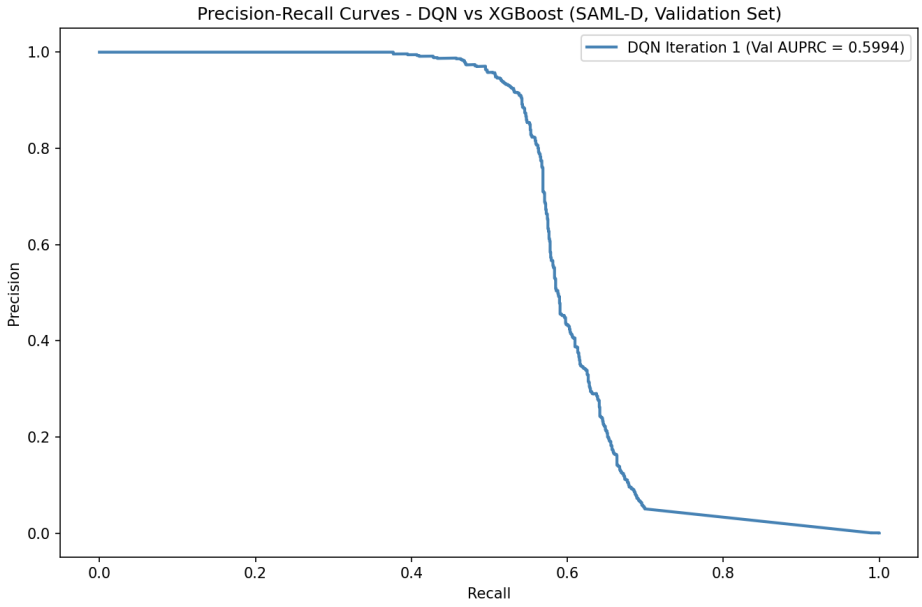

Figure 5: Precision-recall curves on the validation set: DQN Iteration 1 (Val AUPRC = 0.5994).

Iteration 2: Zhinin-Vera et al. reward function ($\lambda = 0.1$) Iteration 2 implements the reward function of Zhinin-Vera et al., who empirically determine $\lambda = 0.1$ as the value producing the best results on a credit card fraud dataset with imbalance ratio $\rho = 0.001728$. The discount factor is set to $\gamma = 0.9$ following Zhinin-Vera et al. (2020). All other hyperparameters are identical to Iteration 1, isolating λ and γ as the only variables that differ between iterations (Table 7). This iteration achieved a validation AUPRC of 0.6203, a marginal improvement over Iteration 1 (0.5994), establishing the Zhinin-Vera et al. reward function as the slightly stronger configuration and serving as the basis for further optimisation in Iteration 3 (Figure 6, Figure 7).

Table 7: DQN Iteration 2 hyperparameters — changes from Iteration 1 highlighted

Parameter	**Value**	**Source**
`gamma` (γ)	0.9	Zhinin-Vera et al. (2020)
`lambda` (λ)	0.1	Zhinin-Vera et al. (2020)
`remaining`	identical	see Table 6

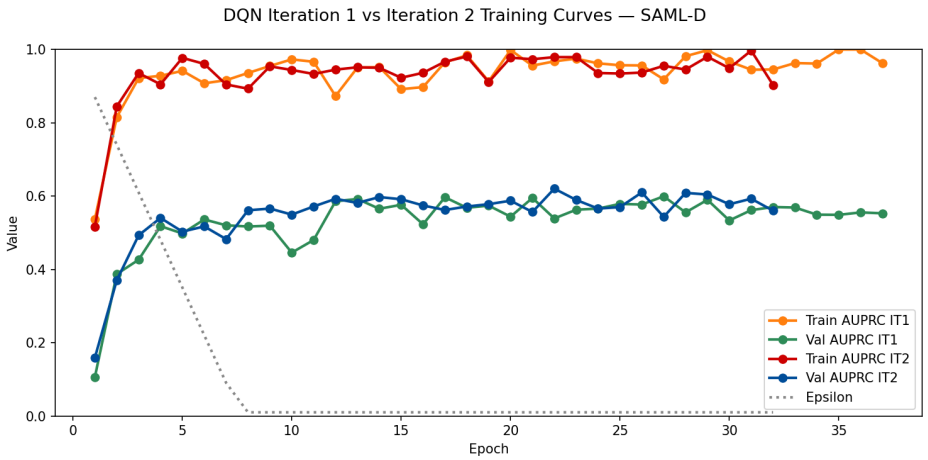

Figure 6: DQN training curves comparison: Iteration 1 ($\lambda = \rho = 0.001039$, $\gamma = 0.1$) vs Iteration 2 ($\lambda = 0.1$, $\gamma = 0.9$). Iteration 2 achieves a higher validation AUPRC of 0.6203 compared to 0.5994 in Iteration 1.

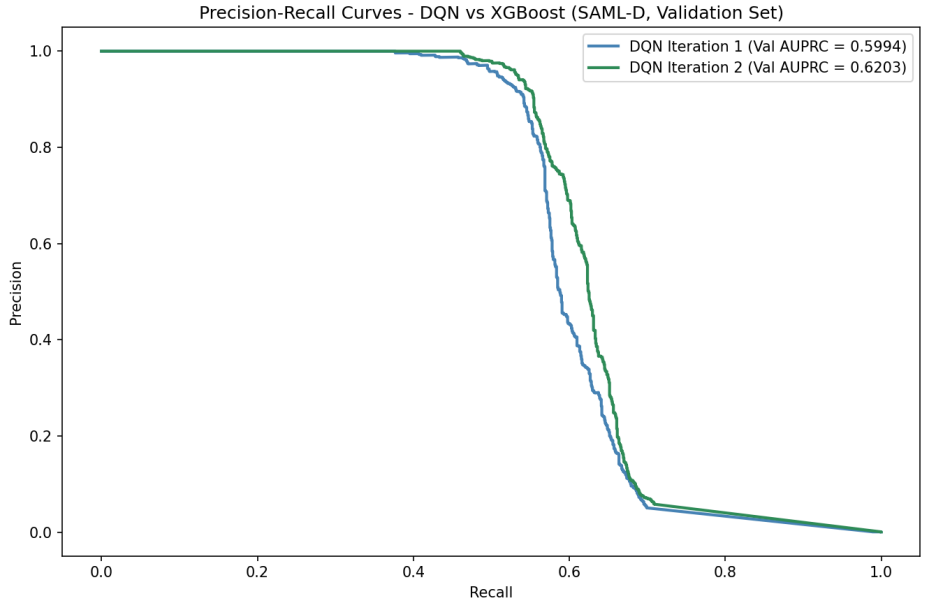

Figure 7: Precision-recall curves on the validation set: DQN Iteration 1 (Val AUPRC = 0.5994), DQN Iteration 2 (Val AUPRC = 0.6203).

Iteration 3: Adjusted epsilon decay Analysis of the Iteration 2 training curves revealed that epsilon decayed to its minimum of 0.01 by epoch 8, leaving the agent in pure exploitation mode for the remaining 29 epochs. During this phase, train AUPRC frequently reached 0.95–1.0 while validation AUPRC plateaued around 0.55–0.62, indicating overfitting from insufficient exploration. At the Iteration 2 decay rate of 0.00001, with epsilon decayed once per 512 transactions (≈12,995 steps per epoch), epsilon reaches its minimum after 99,000/12,995 ≈ 7.6 epochs, consistent with this observation.

Iteration 3 keeps all hyperparameters identical to Iteration 2 but slows epsilon decay to extend exploration to epoch 20 (≈2.5× longer), computed as:

$$\texttt{epsilon\_decay} = \frac{1.0 - 0.01}{12{,}995 \times 20} = 0.0000038$$

This achieved a validation AUPRC of 0.6132, a marginal decline from Iteration 2 (0.6203), suggesting additional exploration time alone does not reduce overfitting on this dataset (Figures 8, 9).

Table 8: DQN Iteration 3 hyperparameters — changes from Iteration 2

Parameter	Value	Source
`epsilon_decay`	0.0000038	computed to reach minimum at epoch 20
`remaining`	identical	see Table 7

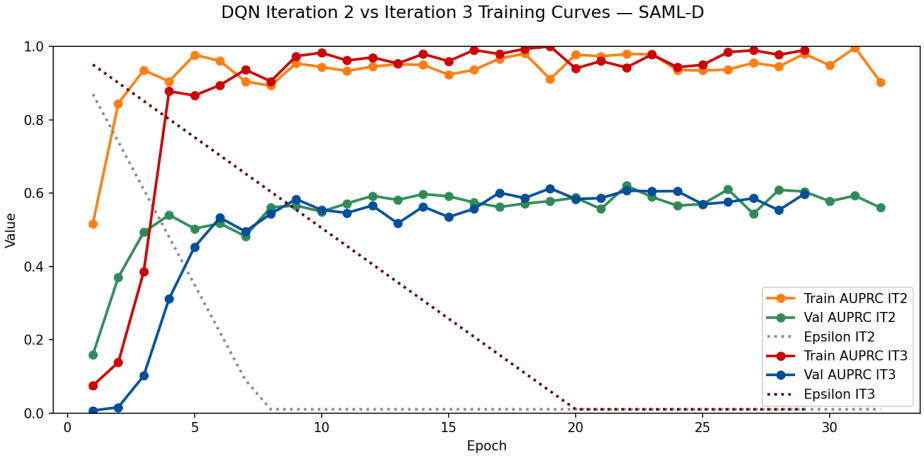

Figure 8: DQN training curves comparison: Iteration 2 vs Iteration 3. Iteration 3 uses a slower epsilon decay (0.0000038 vs 0.00001), extending exploration to epoch 20.

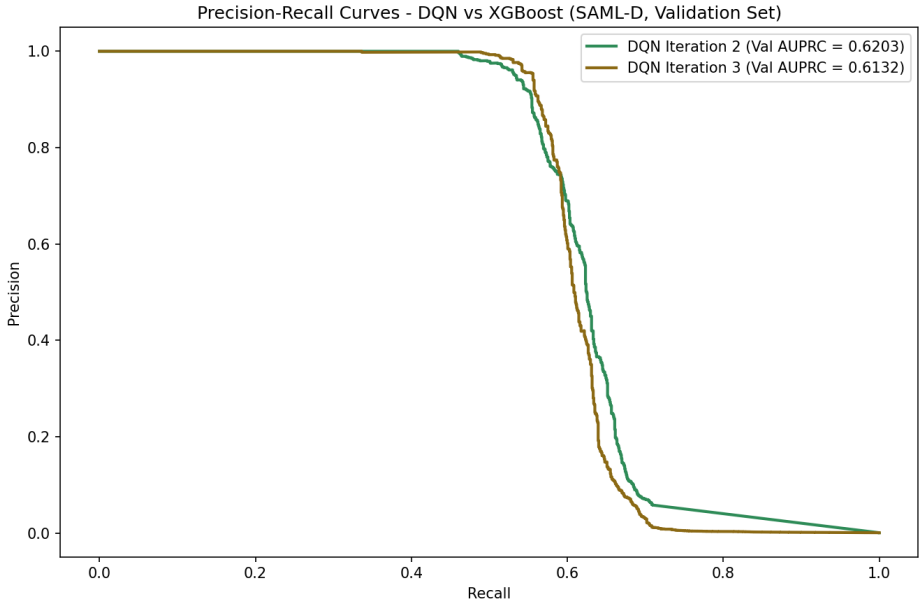

Figure 9: Precision-recall curves on the validation set: DQN Iteration 2 (Val AUPRC = 0.6203), DQN Iteration 3 (Val AUPRC = 0.6132).

Iteration 4: Dropout regularisation An alternative approach to reducing overfitting is dropout regularisation, which randomly disables a fraction of neurons during each forward pass, preventing the

network from over-relying on specific features Srivastava et al. (2014). Since Iteration 3 did not improve upon Iteration 2, Iteration 4 builds directly on Iteration 2, keeping all hyperparameters identical ($\lambda = 0.1$, $\gamma = 0.9$, ε decay $= 0.00001$), and adds dropout to the Q-network following Srivastava et al. (2014), who recommend $p = 0.5$ for hidden units as close to optimal across a wide range of networks and tasks. Dropout is applied to all four hidden layers with $p = 0.5$, but not the input layer: unlike the dense continuous inputs Srivastava et al. tested with $p = 0.8$, the input here is 91 features combining standardised continuous values and sparse one-hot encoded columns, where dropout would risk zeroing out the only active feature in a one-hot group entirely (Table 9).

Table 9: DQN Q-network architecture with dropout (Iteration 4)

Layer	**Neurons**	**Activation**	**Dropout**
Input	91	–	–
Hidden 1	124	ReLU	0.5
Hidden 2	62	ReLU	0.5
Hidden 3	31	Tanh	0.5
Hidden 4	15	Linear	0.5
Output	2	Linear	–

Adding dropout to the hidden layers is the only change from Iteration 2 (Table 7). This iteration achieved a validation AUPRC of 0.5598, a decline compared to Iteration 2 (0.6203), suggesting that $p = 0.5$ might be too aggressive for this network and dataset combination (Figure 10, Figure 11).

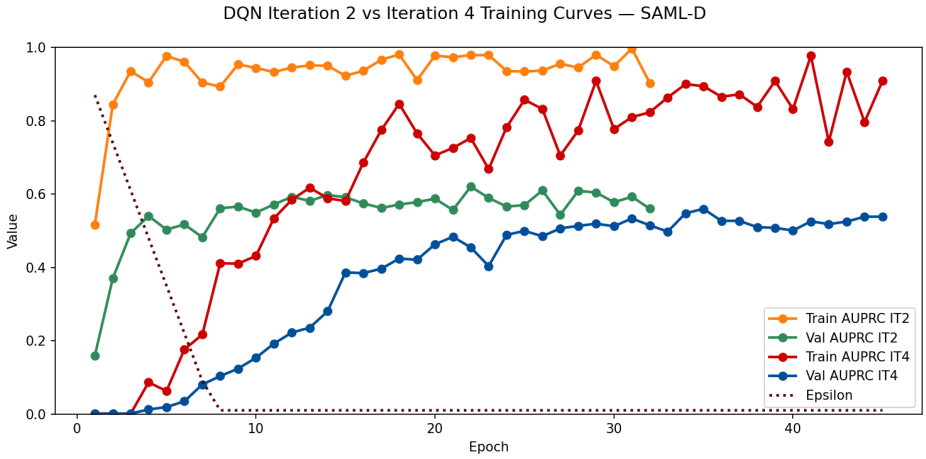

Figure 10: DQN training curves comparison: Iteration 2 vs Iteration 4. Iteration 4 adds dropout ($p = 0.5$) to all hidden layers, reducing the train/val AUPRC gap but also lowering overall performance.

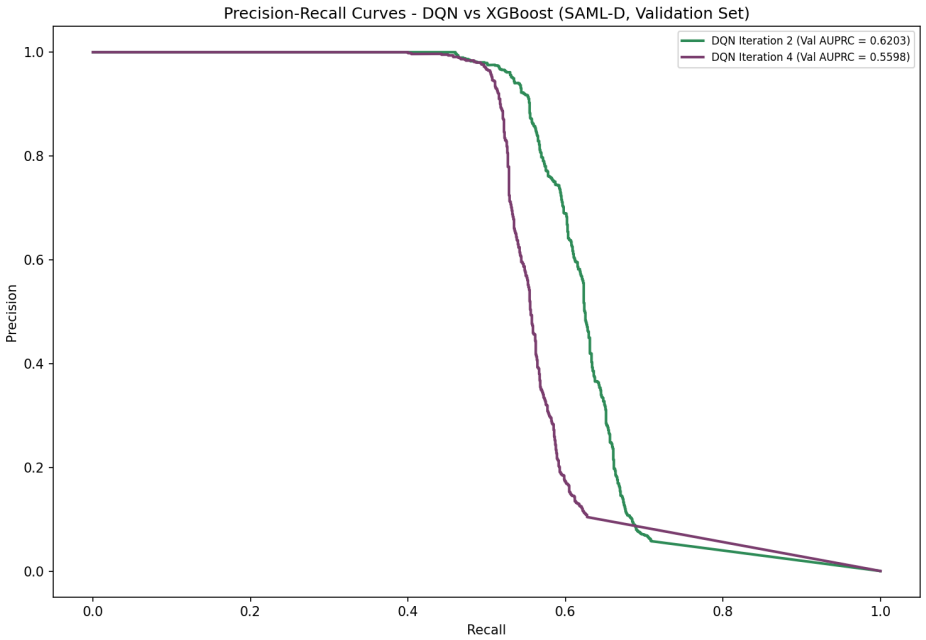

Figure 11: Precision-recall curves on the validation set: DQN Iteration 2 (Val AUPRC = 0.6203), DQN Iteration 4 (Val AUPRC = 0.5598).

Iteration 5: Reduced dropout rate ($p = 0.2$) Iteration 4's dropout of $p = 0.5$ narrowed the train/val AUPRC gap but reduced overall performance (0.6203 to 0.5598), with train AUPRC also dropping significantly. The network lost capacity to learn rather than simply overfitting less. Srivastava et al. Srivastava et al. (2014) note that the optimal dropout rate depends on network size, and their $p = 0.5$ recommendation is derived from large networks with thousands of hidden units; the Q-network here is considerably smaller (124-62-31-15), and the reward signal is already sparse due to SAML-D's 0.104% class imbalance, compounding the difficulty of training under heavy dropout.

Iteration 5 reduces the dropout rate to $p = 0.2$, isolating it as the only variable changed from Iteration 4 (Table 10). This achieved a validation AUPRC of 0.5914, an improvement over Iteration 4 (0.5598) but still below Iteration 2 (0.6203), suggesting dropout regularisation does not benefit this architecture and dataset regardless of rate (Figures 12, 13).

Table 10: DQN Iteration 5 hyperparameters — changes from Iteration 4

Parameter	**Value**	**Source**
`dropout_rate`	0.2	reduced from 0.5; Srivastava et al. (2014)
`remaining`	identical	see Table 9

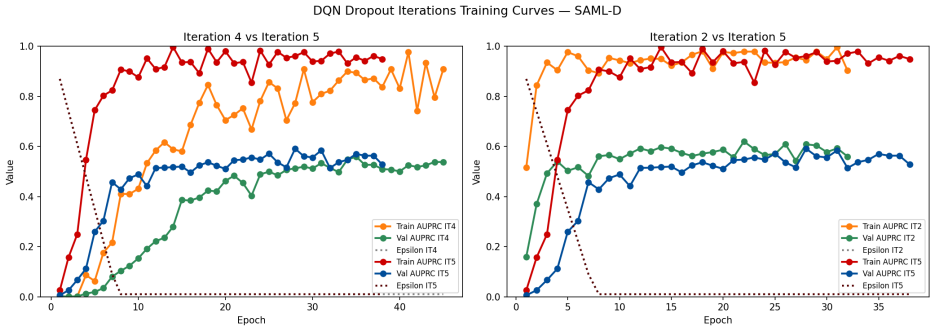

Figure 12: DQN training curves: left panel shows Iteration 4 ($p = 0.5$) vs Iteration 5 ($p = 0.2$); right panel shows Iteration 2 (no dropout) vs Iteration 5 ($p = 0.2$). Reducing dropout from 0.5 to 0.2 partially recovers performance but does not reach Iteration 2 levels.

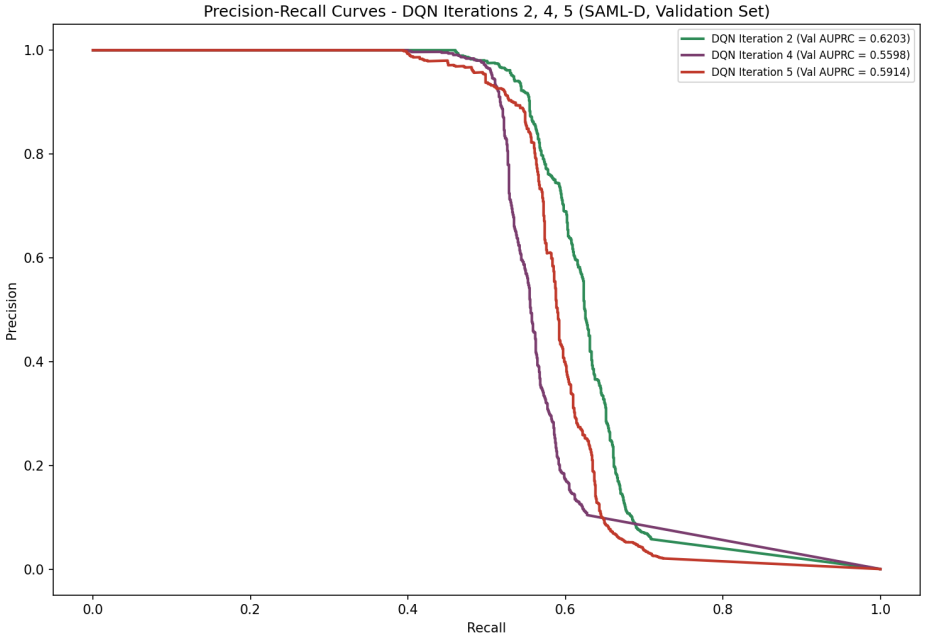

Figure 13: Precision-recall curves on the validation set: DQN Iteration 4 (Val AUPRC = 0.5598) and DQN Iteration 5 (Val AUPRC = 0.5914).

Iteration Summary Table 11 and Figure 14 summarise the validation and test AUPRC across all five DQN iterations.

Iteration 2 outperformed Iteration 1 (0.6203 vs 0.5994 validation AUPRC), establishing the Zhinin-Vera et al. Zhinin-Vera et al. (2020) reward function ($\lambda = 0.1$, $\gamma = 0.9$) as the stronger configuration over Lin et al.'s Lin et al. (2019) formulation ($\lambda = \rho = 0.001039$, $\gamma = 0.1$). Iterations 3 through 5 therefore build on Iteration 2.

Iteration 3 achieves the highest test AUPRC at 0.6691, marginally above Iteration 2 (0.6608), but this 0.0083 difference is negligible given the DQN's run-to-run variability, and no strong conclusions can be drawn about the effect of the adjusted epsilon decay.

Iterations 4 and 5 tested dropout at $p = 0.5$ and $p = 0.2$; neither improved over Iteration 2. Dropout at $p = 0.5$ reduced the train/validation gap but also reduced overall performance, while $p = 0.2$ partially recovered performance without reaching Iteration 2 levels, indicating dropout regularisation does not benefit this architecture and dataset.

Iteration 2 is selected as the final DQN model for comparison against XGBoost, achieving comparable performance to Iteration 3 with a simpler configuration that follows Zhinin-Vera et al. (2020)

more directly.

Table 11: DQN iteration results

Iteration	**Configuration**	**Val AUPRC**	**Test AUPRC**
Iteration 1	Lin et al., $\lambda = \rho$	0.5994	0.6450
Iteration 2	Zhinin-Vera, $\lambda = 0.1$	0.6203	0.6608
Iteration 3	Zhinin-Vera, adjusted ε decay	0.6132	0.6691
Iteration 4	Zhinin-Vera, dropout $p = 0.5$	0.5598	0.5578
Iteration 5	Zhinin-Vera, dropout $p = 0.2$	0.5914	0.6197

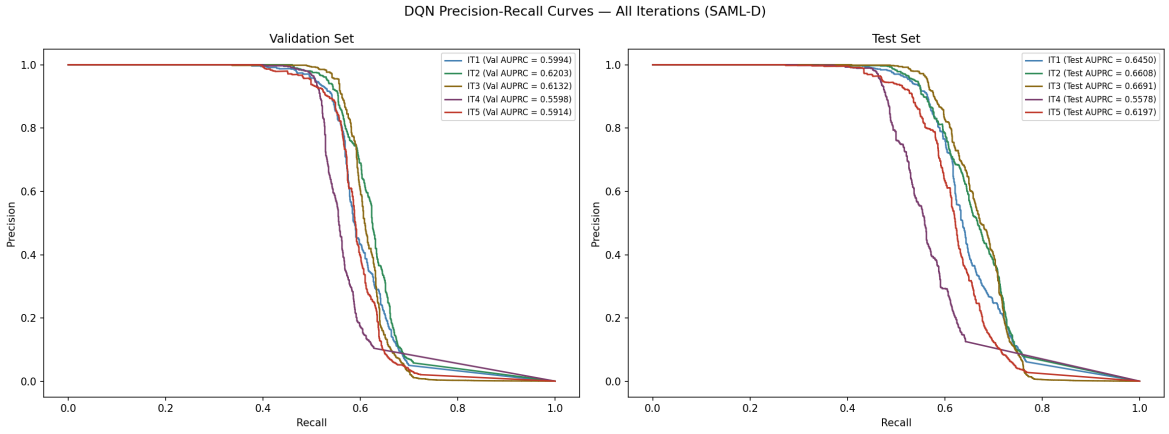

Figure 14: Precision-recall curves for all DQN iterations on the validation set (left) and test set (right). Iteration 2 achieves the highest validation AUPRC (0.6203) while Iteration 3 achieves the highest test AUPRC (0.6691).

4 Results

This chapter presents the results of comparing the best XGBoost model (Iteration 3) against the best DQN model (Iteration 2). Both are evaluated on the following quality criteria: performance, explainability, robustness, model complexity, and resource demand. Scalability is out of scope, as it requires testing across varying data volumes, which this research does not support given its reliance on a single fixed dataset.

4.1 Performance

Table 12 presents the validation and test AUPRC for all iterations of both models. XGBoost Iteration 3 is the best performing XGBoost configuration based on validation and test AUPRC. For the DQN, Iteration 3 achieves the highest test AUPRC at 0.6691, marginally above Iteration 2 at 0.6608. However, given the DQN's run-to-run variability, this difference of 0.0083 is within the noise margin and no strong conclusions can be drawn about which is truly better. Iteration 2 is therefore selected as the final DQN model as it follows the literature more directly without additional modifications.

XGBoost Iteration 3 achieves a test AUPRC of 0.9275, substantially outperforming DQN Iteration 2 at 0.6608, a difference of 0.2667 (Figure 15). Notably, the untuned XGBoost baseline (Iteration 1, 0.9232) already outperforms the best DQN result by 0.2624. Despite five DQN iterations exploring different reward functions, epsilon decay schedules and dropout regularisation, none came close to XGBoost's performance level.

Table 12: AUPRC results on the validation and test set

Model	**Configuration**	**Val AUPRC**	**Test AUPRC**
XGBoost Iteration 1	Defaults	0.9087	0.9232
XGBoost Iteration 2	Grid search	0.9107	0.9260
XGBoost Iteration 3	Bayesian optimisation	**0.9136**	**0.9275**
DQN Iteration 1	Lin et al., $\lambda = \rho$	0.5994	0.6450
DQN Iteration 2	Zhinin-Vera, $\lambda = 0.1$	**0.6203**	0.6608
DQN Iteration 3	Zhinin-Vera, adjusted ε decay	0.6132	**0.6691**
DQN Iteration 4	Zhinin-Vera, dropout $p = 0.5$	0.5598	0.5578
DQN Iteration 5	Zhinin-Vera, dropout $p = 0.2$	0.5914	0.6197

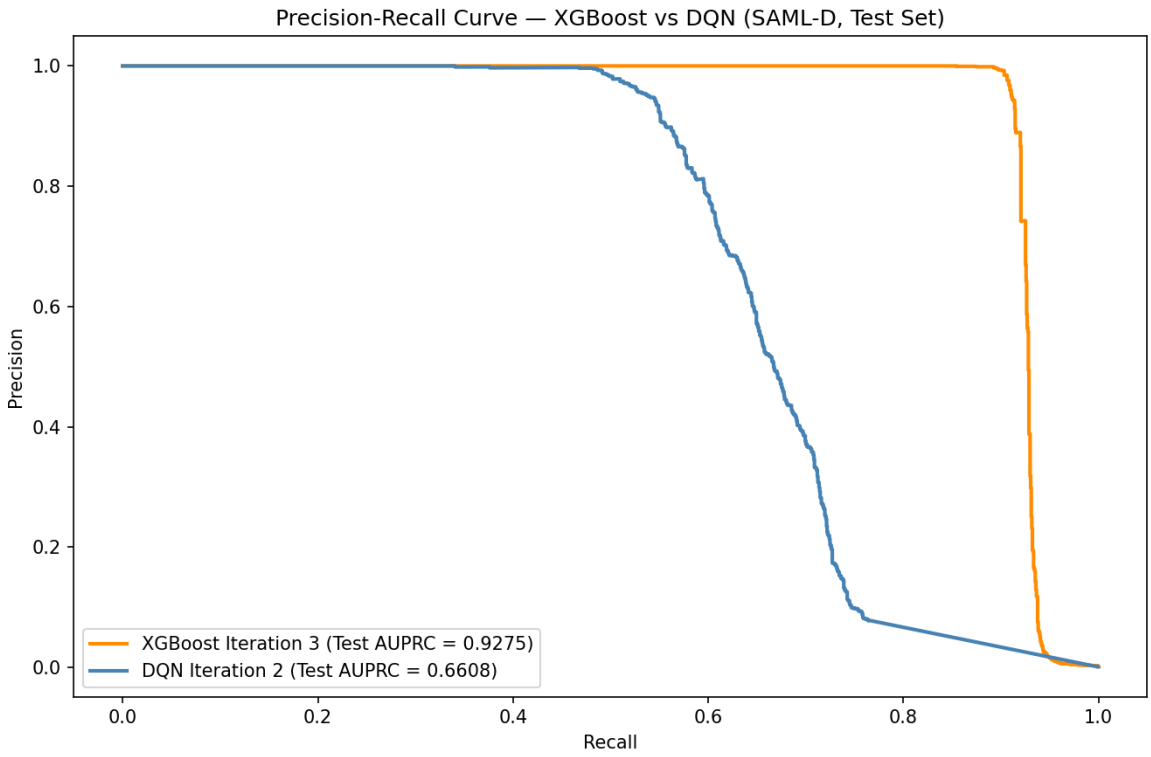

Figure 15: Precision-recall curves on the test set: XGBoost Iteration 3 (Test AUPRC = 0.9275) vs DQN Iteration 2 (Test AUPRC = 0.6608).

4.2 Explainability

SHAP values were computed for both models on a sample of 2,500 test transactions to assess which features drive each model's predictions and to satisfy the explainability requirement imposed by the Wwft Ministerie van Financiën (2008). XGBoost supports exact SHAP values through its tree structure, while the DQN requires an approximation-based approach due to its neural network architecture. One-hot encoded categorical features were aggregated back to their original feature names before plotting to improve interpretability.

4.2.1 Global Feature Importance

XGBoost - Bar Plot `pair_tx_count` dominates the XGBoost importance ranking with a mean absolute SHAP value of 5.52, nearly three times the second-ranked feature `Payment_type` at 1.85. The remaining features form a long tail descending from 1.04. On average across all transactions, `pair_tx_count` is the leading indicator, though individual transactions may be driven by different features as shown in the waterfall analysis 4.2.2.

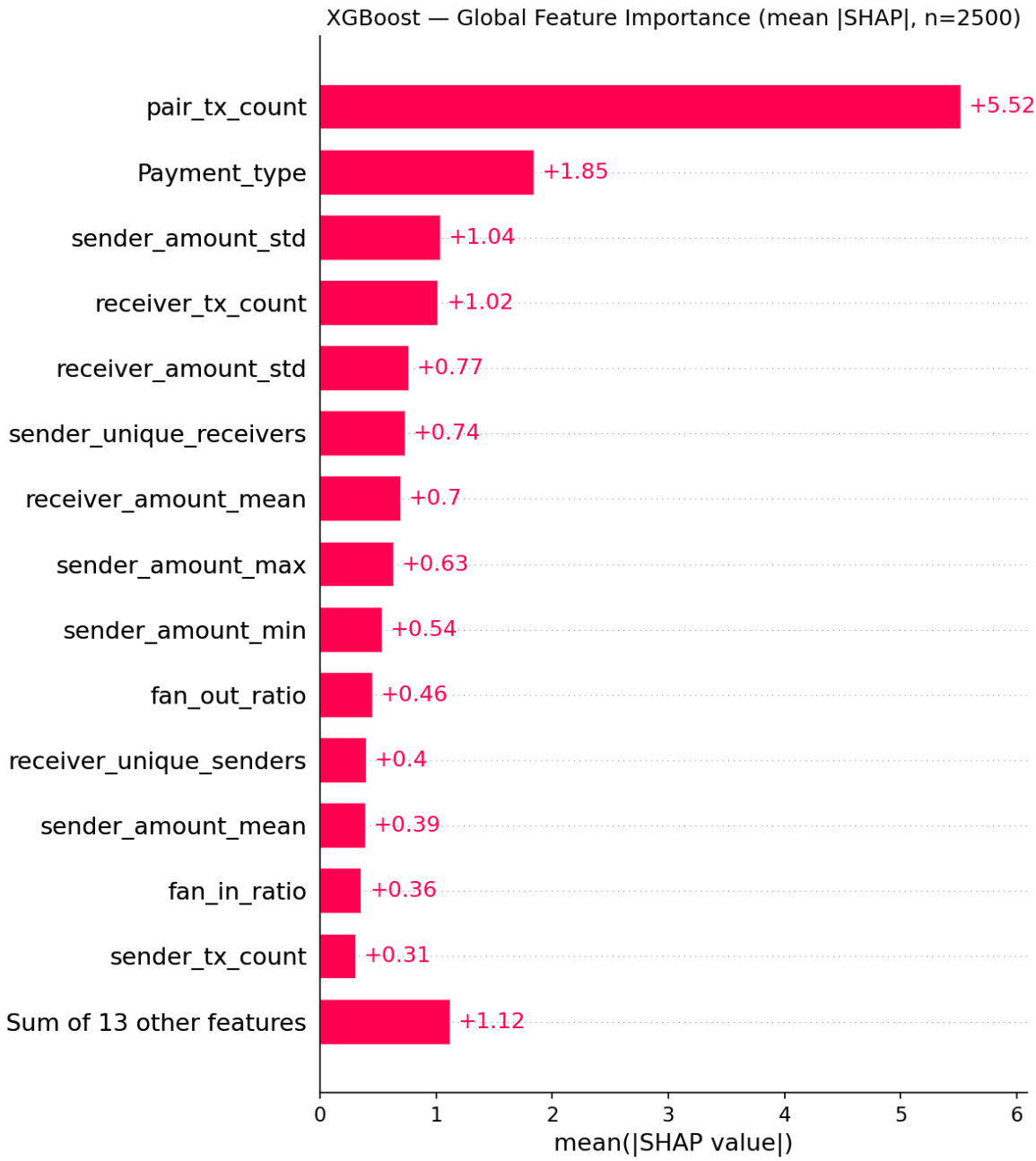

Figure 16: XGBoost global feature importance (mean absolute SHAP value, n=2,500).

DQN - Bar Plot The DQN importance scale reaches only 0.02 for the top-ranked feature `receiver_unique_senders`, with most remaining features contributing around 0.01. This is two orders of magnitude smaller than XGBoost, indicating that individual features have far less impact on the DQN's output. Notably, `pair_tx_count`, the dominant feature in XGBoost, ranks fifth with near-zero importance. The sum of the 13 remaining features at 0.03 is the largest bar in the plot, suggesting importance is spread thinly across many features rather than concentrated in a meaningful signal (Figure 17).

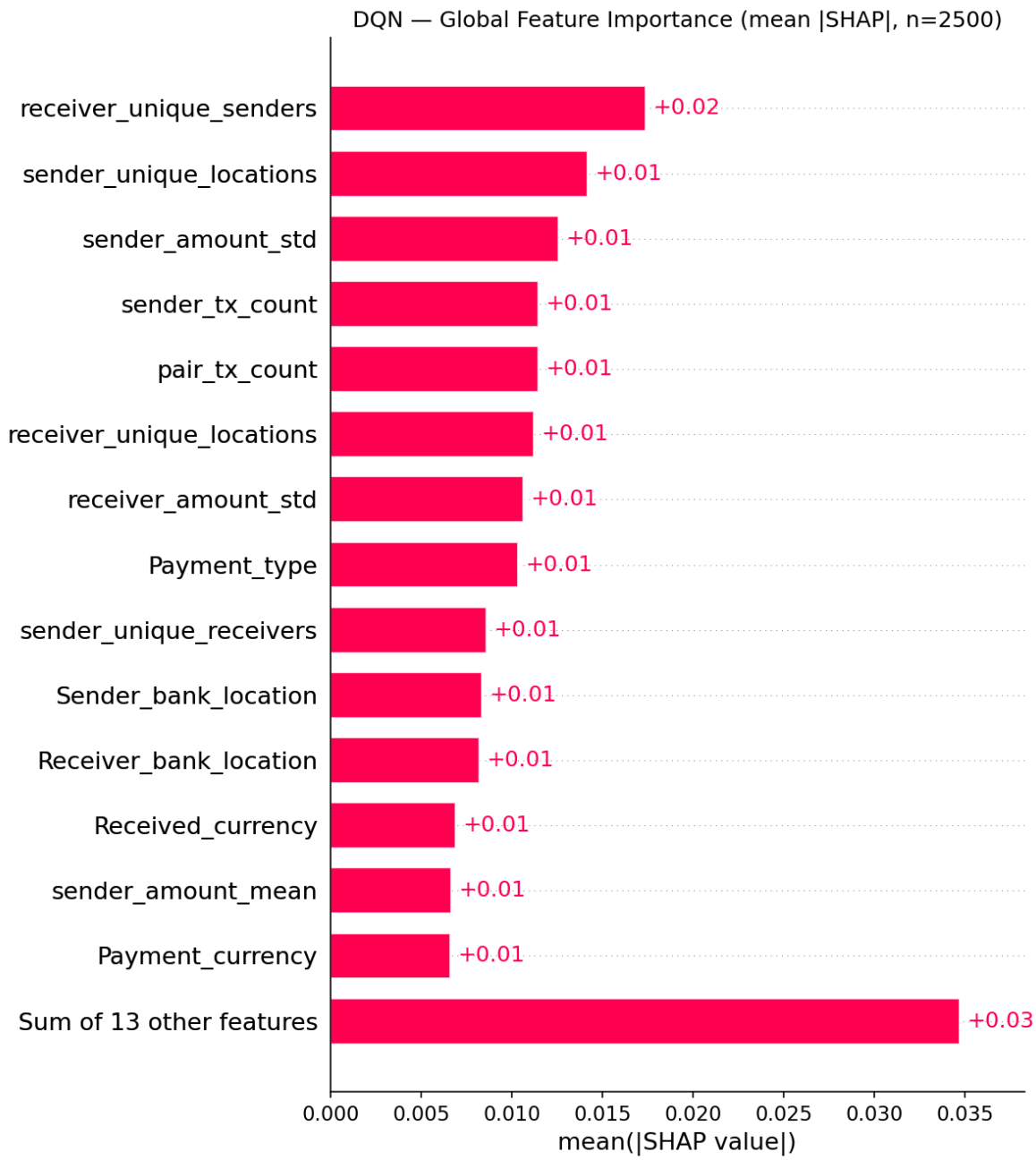

Figure 17: DQN global feature importance (mean absolute SHAP value, n=2,500).

XGBoost - Beeswarm The beeswarm shows that the lowest `pair_tx_count` values, close to 0, are all on the right side pushing toward suspicious. Once the value reaches roughly 25–30% of the color scale, the feature flips entirely to the left side pushing toward normal. The transition is clean, making `pair_tx_count` the most decisive feature in the model.

Since `Payment_type` is a categorical feature, each distinct color cluster in the beeswarm represents a different payment type rather than a continuous value range. Certain payment types are associated with higher suspicion scores, while others push toward normal predictions. `Payment_type` shows that certain payment types are associated with higher suspicion scores. `sender_amount_std`, `receiver_tx_count`, `receiver_amount_std`, `sender_amount_min` and `receiver_amount_mean` primarily contribute to normal predictions, with low values of these features pushing toward normal. The remaining features show no clear patterns (Figure 18).

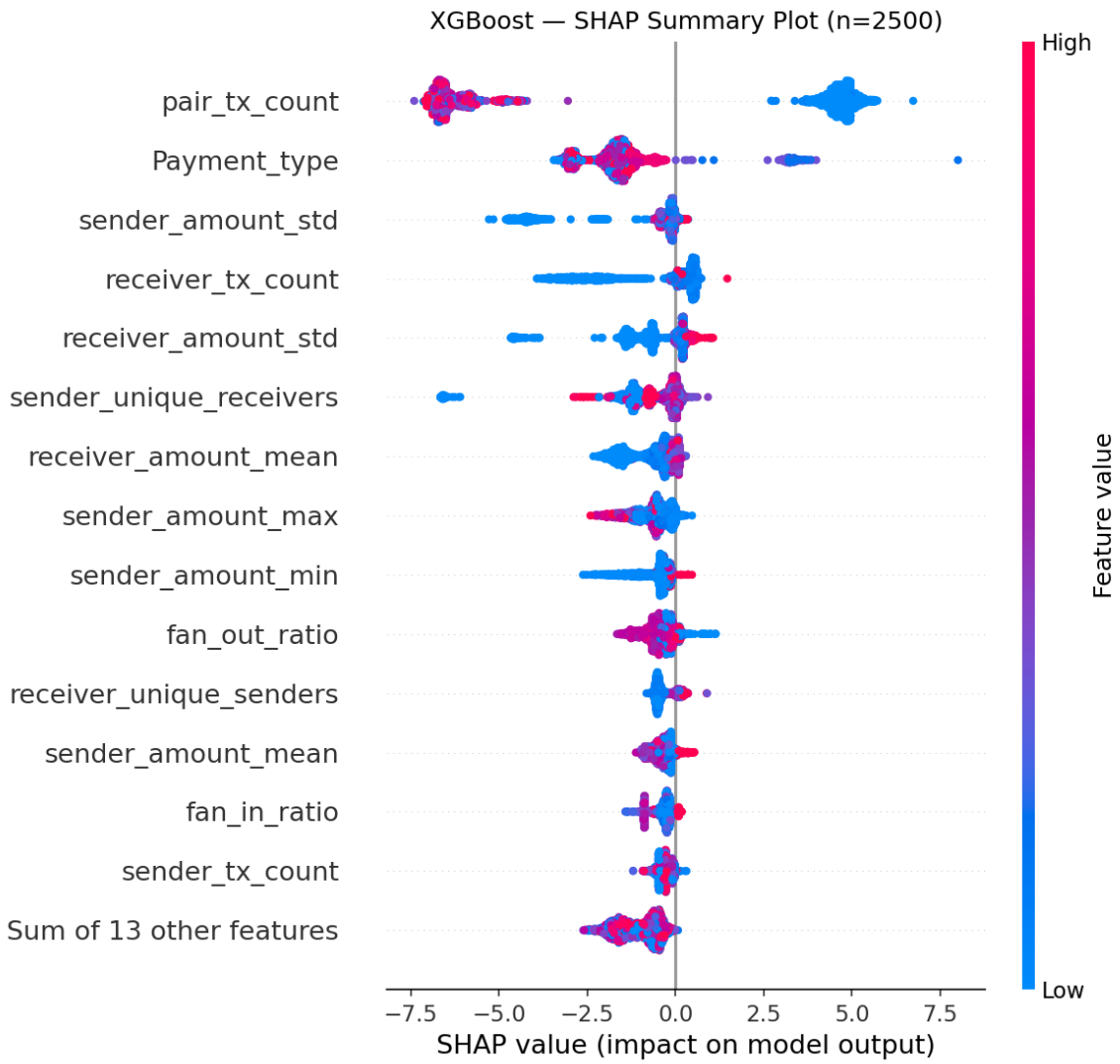

Figure 18: XGBoost SHAP beeswarm plot (n=2,500).

DQN - Beeswarm The DQN beeswarm looks very different from the XGBoost beeswarm. Almost all points cluster between -0.1 and $+0.1$, with a large mass sitting at zero. For the top-ranked features, high values (red) are consistently grouped together providing a directional push, while low values (blue) sit at zero or on the opposite side providing no clear signal. This inconsistency means low feature values do not reliably indicate either class, making the DQN's decision boundary less predictable than XGBoost's. Medium values (purple) show almost no contribution in either direction, clustering entirely at zero. All contributions are compressed within a narrow range, two orders of magnitude smaller than XGBoost, meaning no feature produces a meaningful impact on the model output regardless of its value (Figure 19).

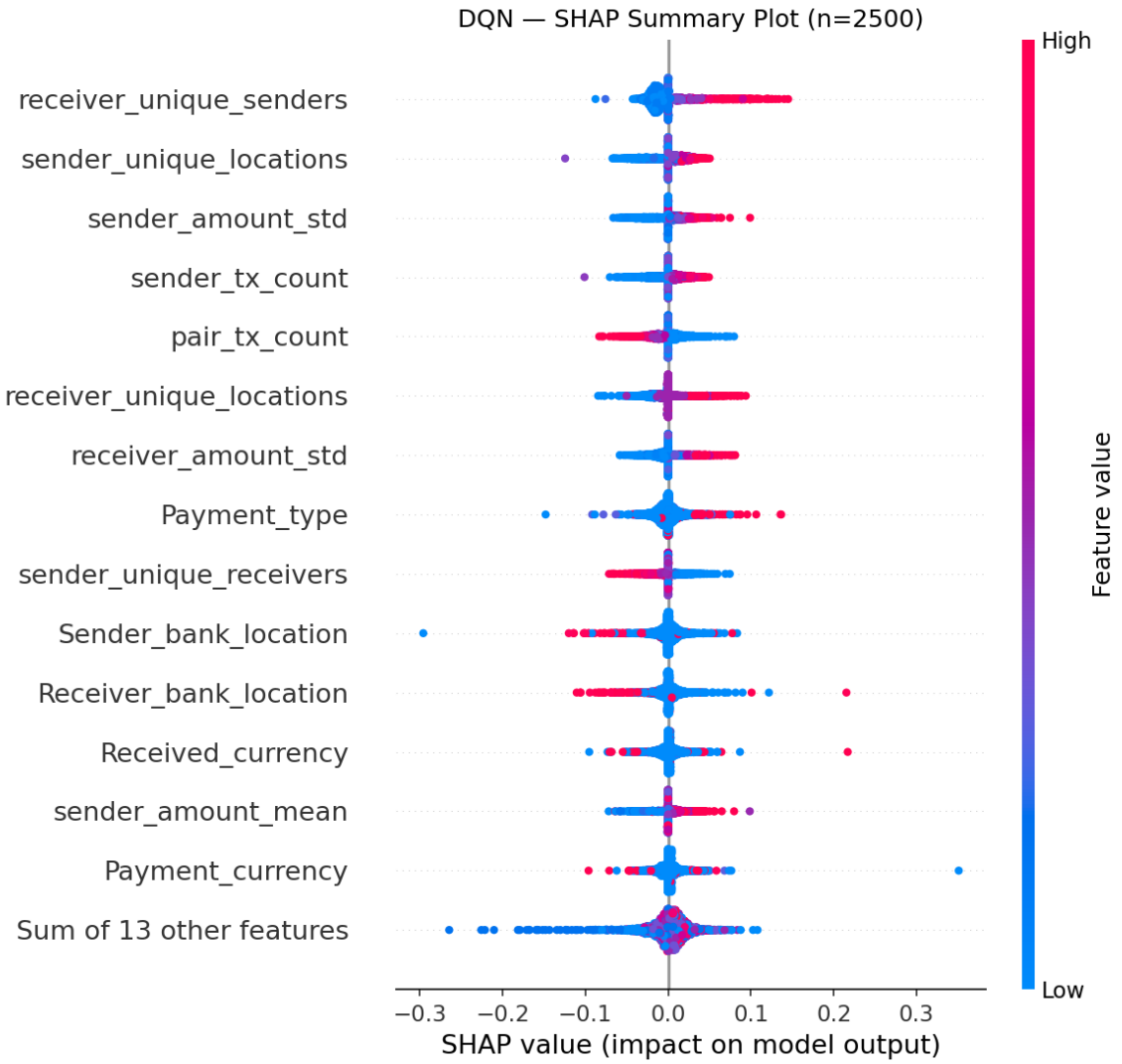

Figure 19: DQN SHAP beeswarm plot (n=2,500).

4.2.2 Individual Transaction Analysis

Eight transactions were selected from the test set, four laundering and four normal, and analysed using SHAP waterfall plots for both models. The same transactions are used for both models to enable direct comparison. The plots show the raw suspiciousness score output by each model without applying a classification threshold, as the optimal threshold depends on operational context and is not fixed in this research.

XGBoost Two distinct detection pathways are visible across the five laundering transactions. In tx6024 (score 0.9998) and tx1986 (score 0.9972), `pair_tx_count` = 0.0 is the dominant driver at +6.83 and +6.15 respectively, confirming what the beeswarm showed: a new sender-receiver relationship is the primary suspicious signal.

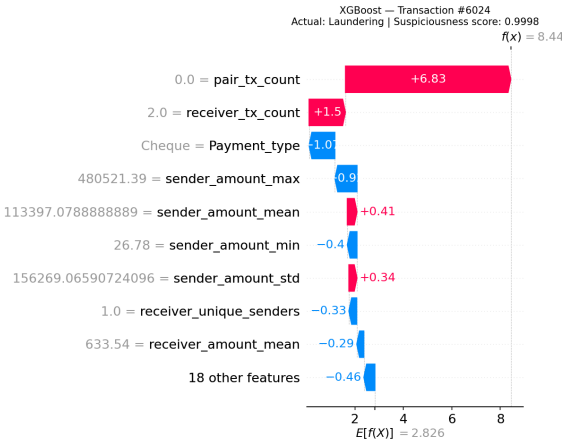

Figure 20: XGBoost SHAP waterfall for tx6024 (Laundering, score 0.9998).

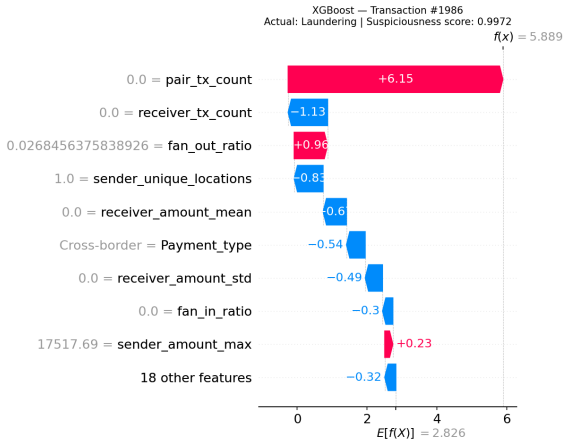

Figure 21: XGBoost SHAP waterfall for tx1986 (Laundering, score 0.9972).

In tx2871 (score 0.9998) and tx3224 (score 0.9999), the pattern shifts entirely. `Cash_Deposit` and `Cash_Withdrawal` become the dominant drivers at +7.97 and +7.70 respectively, while `pair_tx_count` at values of 10–12 actually pulls back negatively. The model has learned that cash-based payment instruments are highly suspicious regardless of relationship history.

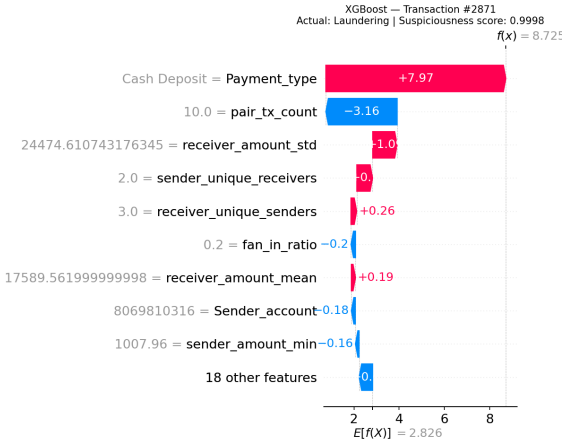

Figure 22: XGBoost SHAP waterfall for tx2871 (Laundering, score 0.9998).

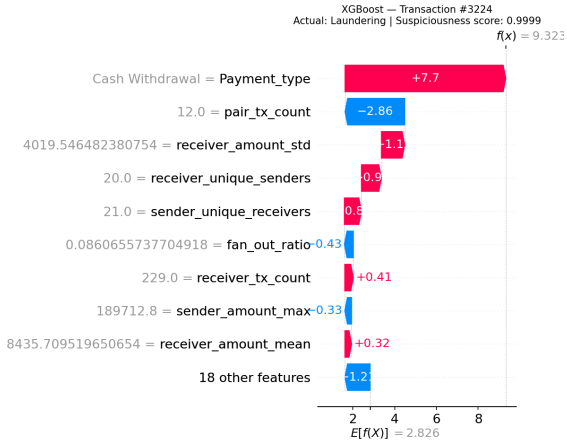

Figure 23: XGBoost SHAP waterfall for tx3224 (Laundering, score 0.9999).

For normal transactions, all four receive a score of 0.0001, demonstrating XGBoost's decisive separation from suspicious cases. `pair_tx_count` values of 8–12 drive large negative contributions in tx0, tx2 and tx3, with credit card and ACH payment types adding further negative push. In tx1, `pair_tx_count` = 0.0 initially pushes toward suspicious, but `receiver_tx_count` = 0.0 immediately counters, and the accumulation of negative contributions from amount statistics and payment type drags the score down to 0.0001. This is a key finding: XGBoost does not mechanically flag every transaction with `pair_tx_count` = 0 as suspicious. It consults the full feature context before making a decision.

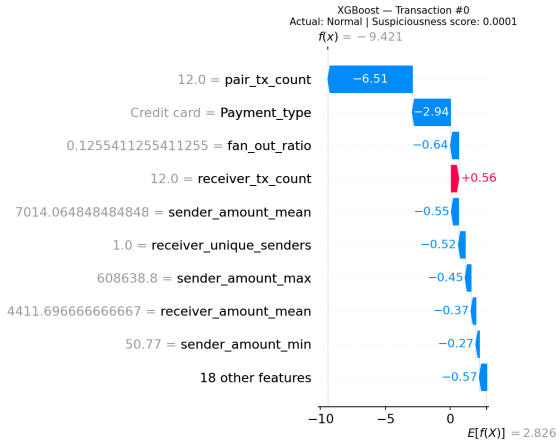

Figure 24: XGBoost SHAP waterfall for tx0 (Normal, score 0.0001).

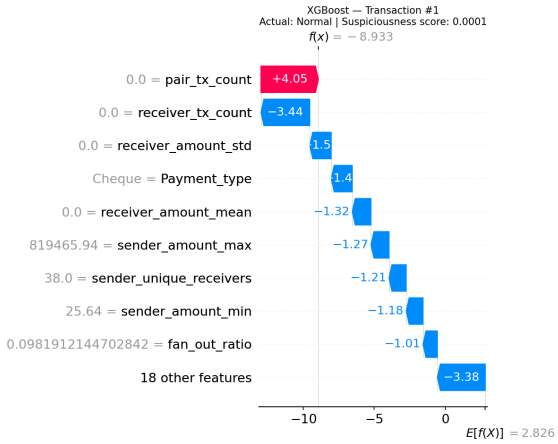

Figure 25: XGBoost SHAP waterfall for tx1 (Normal, score 0.0001).

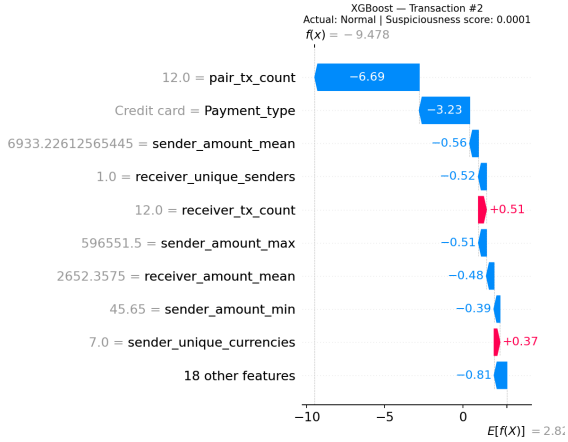

Figure 26: XGBoost SHAP waterfall for tx2 (Normal, score 0.0001).

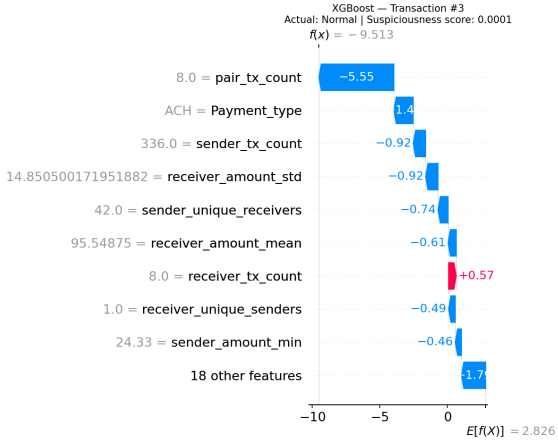

Figure 27: XGBoost SHAP waterfall for tx3 (Normal, score 0.0001).

DQN The DQN only achieves meaningful suspiciousness scores when a cash-based payment type is present. tx2871 (score 0.7769) is driven by `Cash_Deposit` contributing +0.19. tx3224 (score 0.8606) is similarly driven by `Cash_Withdrawal` at +0.16.

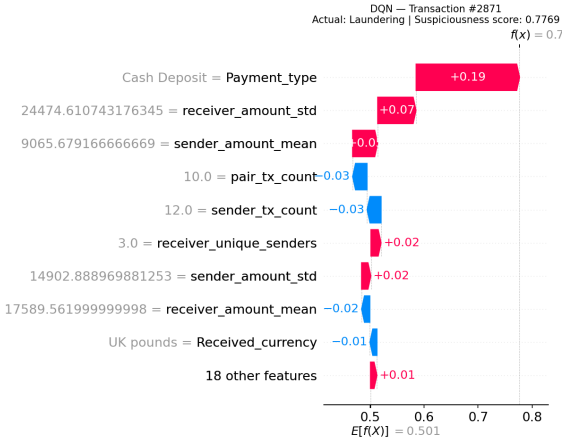

Figure 28: DQN SHAP waterfall for tx2871 (Laundering, score 0.7769).

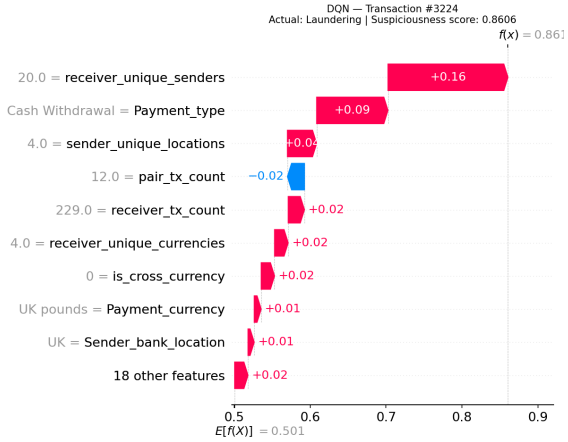

Figure 29: DQN SHAP waterfall for tx3224 (Laundering, score 0.8606).

In contrast, tx6024 and tx1986, which XGBoost scores at 0.9998 and 0.9972, receive DQN scores of only 0.5471 and 0.5007. For tx1986 specifically, the contributions cancel out almost perfectly, with every feature contributing between -0.03 and $+0.03$, resulting in a score indistinguishable from random.

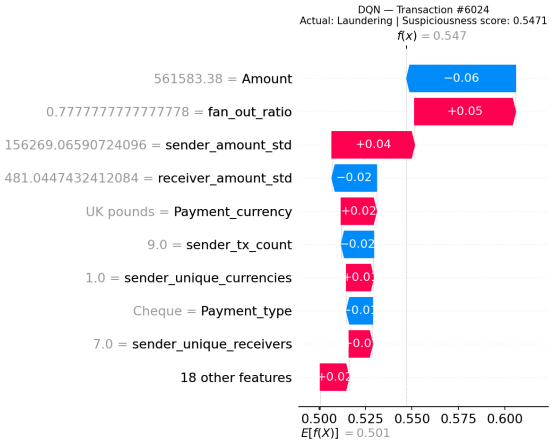

Figure 30: DQN SHAP waterfall for tx6024 (Laundering, score 0.5471).

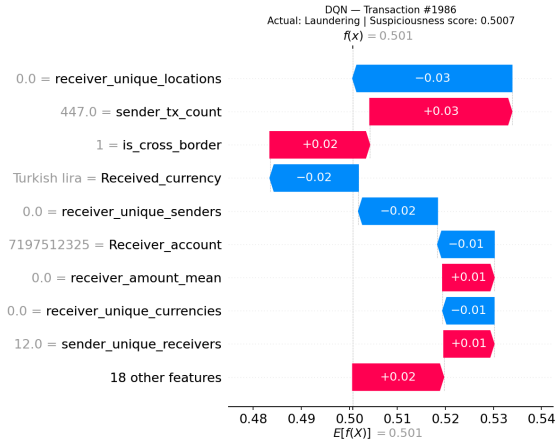

Figure 31: DQN SHAP waterfall for tx1986 (Laundering, score 0.5007).

All four normal transactions score exactly 0.5007. Feature contributions are between -0.03 and $+0.03$ and cancel each other out in every case. The DQN does not differentiate normal transactions from each other, nor from some laundering transactions, producing scores that are practically indistinguishable from random guessing.

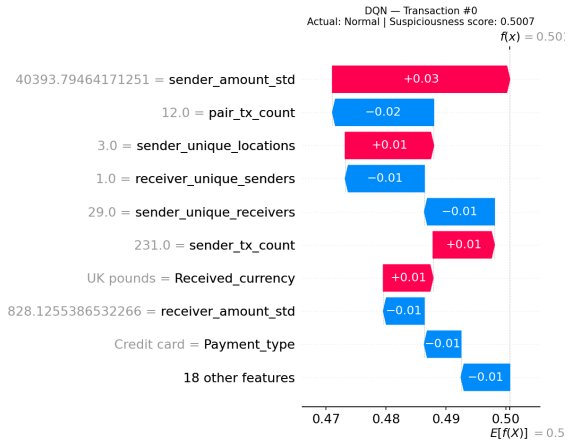

Figure 32: DQN SHAP waterfall for tx0 (Normal, score 0.5007).

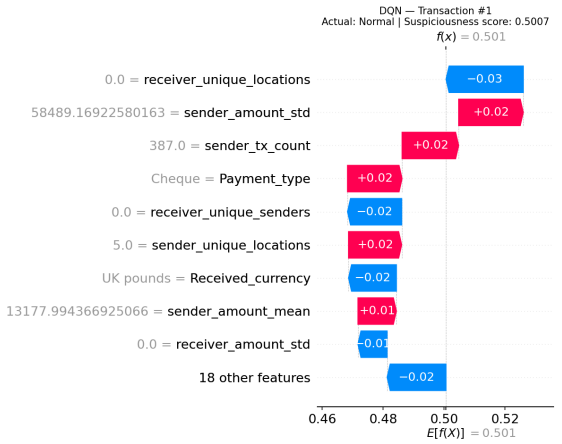

Figure 33: DQN SHAP waterfall for tx1 (Normal, score 0.5007).

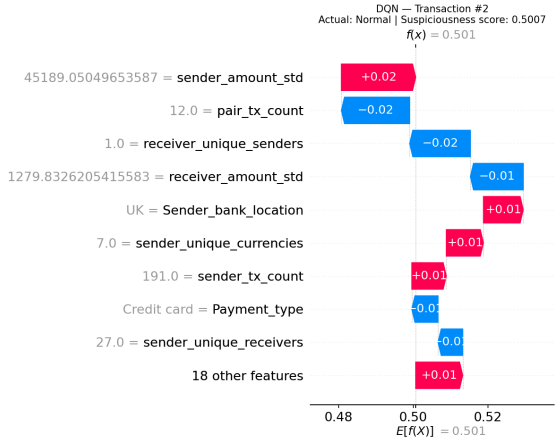

Figure 34: DQN SHAP waterfall for tx2 (Normal, score 0.5007).

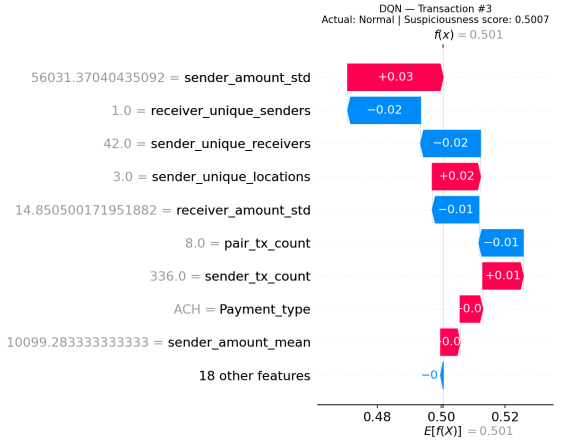

Figure 35: DQN SHAP waterfall for tx3 (Normal, score 0.5007).

Comparison An interesting case is tx3224 (Figures 36 37), where both models correctly identify the transaction as suspicious, but through different reasoning. XGBoost scores it at 0.9999, driven primarily by `Cash_Withdrawal` at +7.70, with `pair_tx_count` pulling back. The DQN scores it at 0.8606, also driven by `Cash_Withdrawal`, but with a much smaller contribution of +0.16. This is the only case where both models rely on the same dominant feature, yet the confidence gap remains large.

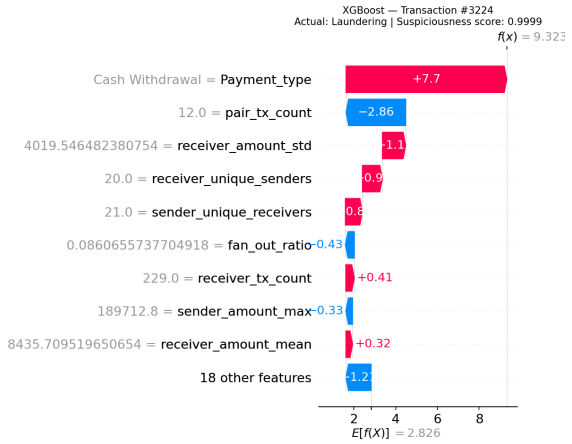

Figure 36: XGBoost SHAP waterfall for tx3224 (Laundering, score 0.9999).

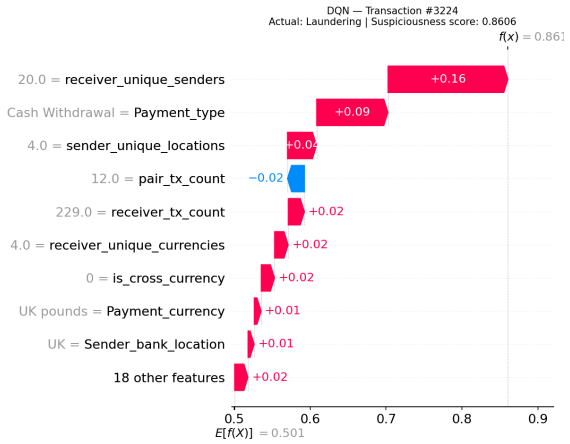

Figure 37: DQN SHAP waterfall for tx3224 (Laundering, score 0.8606).

Transaction tx1 (Figures 38 39) shows a case for a normal transaction. Despite `pair_tx_count` = 0.0, which would typically push toward suspicious, XGBoost correctly scores it at 0.0001. The model consults additional features: `receiver_tx_count` = 0.0 counters the pair count signal, and the accumulation of the remaining features override it entirely. The DQN scores the same transaction at 0.5007, showing no ability to resolve the ambiguity.

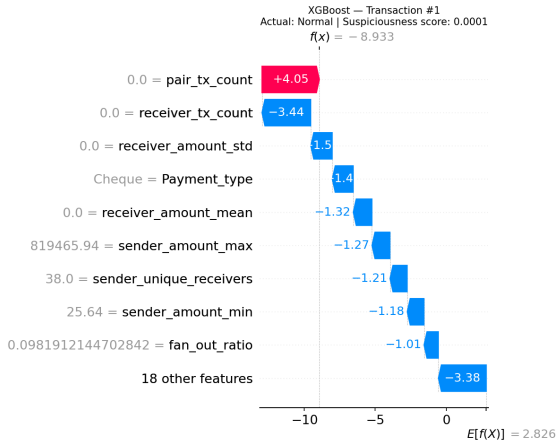

Figure 38: XGBoost SHAP waterfall for tx1 (Normal, score 0.0001).

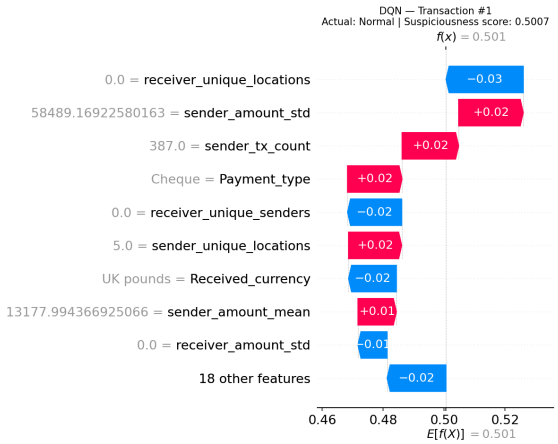

Figure 39: DQN SHAP waterfall for tx1 (Normal, score 0.5007).

4.3 Robustness

Hyperparameter Sensitivity XGBoost was largely insensitive to hyperparameter choices: across 150 grid search combinations and 100 Bayesian optimisation trials, validation AUPRC improved only marginally from 0.9087 to 0.9136 (test: 0.9232 to 0.9275), suggesting near-optimal performance with defaults. The DQN was more sensitive: switching the reward function alone moved validation AUPRC from 0.5994 to 0.6203, and adding dropout at $p = 0.5$ caused a decline to 0.5598. Single parameter changes consistently produced noticeable shifts in contrast to XGBoost's stability.

Reproducibility XGBoost is fully deterministic, producing identical results on every run. The DQN exhibits run-to-run variability of approximately ±0.03 AUPRC even with a fixed seed, due to the stochastic nature of experience replay, epsilon-greedy exploration, and weight initialisation.

Score Distribution XGBoost produces decisive scores: laundering transactions scored between 0.9972 and 0.9999, normal transactions between 0.0001 and 0.0002, leaving little ambiguity. The DQN's scores are far more compressed: laundering transactions scored between 0.5007 and 0.8606 (higher only with cash-based payment types), while all normal transactions scored exactly 0.5007, indistinguishable from the lowest-scoring laundering cases. This leaves little basis for prioritising alerts.

4.4 Model Complexity

Table 13 provides a structured comparison of the two models across key complexity dimensions. The following paragraphs discuss each criterion in detail.

XGBoost operates directly on 27 features, handling categorical variables natively. The DQN requires 91 features after one-hot encoding the five categorical columns, a technical necessity since the DQN cannot process categorical inputs directly; numeric features were also standardised, adding a preprocessing step XGBoost does not require.

XGBoost reaches near-optimal performance with default hyperparameters (Iteration 1: test AUPRC 0.9232), consistent with Probst et al. (2019)'s finding that XGBoost has few impactful hyperparameters; extensive tuning across 150 grid search combinations and 100 Bayesian optimisation trials added only 0.0049.

The DQN has no established defaults for its core parameters (reward function, discount factor, epsilon decay, replay buffer size, batch size, target update frequency, architecture), none standardised across the literature. Performance differences between configurations were small and difficult to attribute given run-to-run variability, making the DQN considerably more complex to optimise.

A further difference lies in the training objective. XGBoost directly optimises AUPRC through its `eval_metric=aucpr` setting, meaning every boosting round explicitly tries to improve the metric used for evaluation. The DQN does not optimise AUPRC directly. Instead, it optimises Q-values through the Bellman equation, shaped by a reward function designed to encourage correct classification. AUPRC is only computed post-hoc on the validation set after each epoch.

XGBoost's tree structure allows exact SHAP values to be computed efficiently via TreeExplainer, making feature attributions reliable and reproducible. The DQN requires KernelExplainer, which produces approximate SHAP values through sampling and is considerably slower and less precise.

Table 13: Model complexity comparison

Criterion	**XGBoost Iteration 3**	**DQN Iteration 2**
Input dimensionality	27 features	91 features
Preprocessing required	None	One-hot encoding + StandardScaler
Hyperparameters tuned	4 (Table 29)	11 (Table 7) + reward function design
Training objective	Directly optimises AUPRC	Indirectly optimizes AUPRC
Explainability method	Exact SHAP values	Approximate SHAP values

4.5 Resource Demand

Table 14 shows the training time for all iterations of both models. XGBoost completes each iteration in under 32 seconds once hyperparameters are determined, with tuning adding a one-time cost of 56 minutes (grid search) and 38 minutes (Bayesian optimisation). The DQN requires 30 to 45 minutes per iteration regardless of configuration, with no comparable fast tuning phase since each configuration change requires a full training run.

Despite its training speed disadvantages, the DQN's notable advantage is its ability to continue training from existing weights when new data arrives, unlike XGBoost, which requires a full retrain as boosted trees cannot be updated incrementally. However, given XGBoost's training time of under 32 seconds per iteration, even a full retrain remains considerably faster than a single DQN incremental update.

Table 14: Training time per iteration

Model	Tuning time	Training time	Epochs / rounds	Avg per epoch / round
XGBoost Iteration 1	–	31.7s	63 rounds	0.50s
XGBoost Iteration 2	56m 59s	29.8s	41 rounds	0.73s
XGBoost Iteration 3	38m 7s	25.7s	35 rounds	0.73s
DQN Iteration 1	–	41m 0s	37 epochs	66.5s
DQN Iteration 2	–	39m 48s	32 epochs	74.7s
DQN Iteration 3	–	36m 54s	29 epochs	76.4s
DQN Iteration 4	–	63m 6s	45 epochs	84.1s
DQN Iteration 5	–	46m 12s	38 epochs	73.0s

A significant memory difference exists during training. The DQN maintains a replay buffer storing all 6,653,396 training transactions in RAM simultaneously, requiring several gigabytes of memory. XGBoost builds trees sequentially and does not require the full dataset to be held in memory at once, making it considerably more memory-efficient during training.

Regarding model size, the XGBoost model is stored as a compact JSON file of 304 KB, encoding 35 decision trees. The DQN is stored as a PyTorch state dictionary of 88 KB, encoding 19,339 weights and biases.

Table 15 shows the inference and SHAP computation times for the final selected models. Interestingly, the DQN is faster at inference than XGBoost. However, the DQN's SHAP computation is considerably slower: generating an explanation for a single transaction takes 0.391s compared to 0.042s for XGBoost. This difference becomes significant in a production AML system where investigators require real-time explanations for flagged transactions under the Wwft Ministerie van Financiën (2008). The global feature importance computation further illustrates this gap: XGBoost computes SHAP importance across 2,500 transactions in 0.25 seconds, while the DQN requires 14 minutes and 31 seconds for the same computation.

Table 15: Inference and explainability times

Model	Single inference	1,000,000 inf.	SHAP single tx	SHAP Feat. Importance (n=2500)
XGBoost IT3	13.14 ms	0.4970s	0.042s	0.25s
DQN IT2	0.97 ms	0.4170s	0.391s	14m 31.04s

5 Discussion

5.1 Evaluation of Results

Performance in Industry Context XGBoost substantially outperforms the DQN across all evaluation dimensions, with a test AUPRC of 0.9275 compared to 0.6608. To contextualise what these scores mean in practice, the operating threshold required to achieve ING's target recall of 0.95 Attard and de Bruijn (2026) was computed for both models (Table 16).

Table 16: Operating point at recall = 0.95

Model	Threshold	Recall	Precision	FP per TP
XGBoost IT3	0.074	0.9499	0.0161	61
DQN IT2	0.5007	0.9982	0.0012	835

XGBoost reaches a recall of 0.9499 at a threshold of 0.074, producing 61 false positives per true positive. While this false positive rate is high, it reflects the operational reality of extreme class imbalance acknowledged by both ING and ABN AMRO Attard and de Bruijn (2026); Menon and Nguyen (2026). The DQN tells a fundamentally different story. At a threshold of 0.5007, barely above the midpoint of the score range, the model achieves a recall of 0.9982 but at a precision of only 0.0012, generating 835 false positives per true positive. This threshold is not a meaningful decision boundary, as it reflects that nearly all transactions receive a score hovering around 0.5, confirming the DQN has no meaningful discrimination between classes. At any operationally viable threshold, the DQN would either miss the vast majority of laundering cases or flood investigators with an unworkable volume of false alarms.

Performance in Literature Context Direct performance comparisons with existing literature are limited, as no prior work applies reinforcement learning to money laundering detection on tabular transaction data. This research therefore establishes a new baseline for RL-based AML detection rather than extending prior work.

5.2 Evaluation of Design Choices and Iterations

DQN Network Architecture The DQN Q-network architecture was adopted directly from Qayoom et al. Qayoom et al. (2024), whose network was designed for a training set of 248 samples derived from a rolling time window of 30 transactions per cardholder. SAML-D's training set of 6,653,396 transactions is several orders of magnitude larger and covers 28 distinct laundering typologies rather than individual cardholder behaviour. Whether an architecture optimised for 248 samples is appropriate for a dataset of this scale and diversity is an open question. A larger network with greater capacity may have been better suited to the complexity of SAML-D, and this represents a direction for future work.

DQN Iteration Selection Zhinin-Vera et al.'s reward function was selected over Lin et al.'s for further iteration based on a higher validation AUPRC (0.6203 vs 0.5994). Across three repeated runs, Lin et al.'s formulation outperformed Zhinin-Vera et al. only once and by a negligible margin, suggesting Zhinin-Vera et al. is the more consistently performing configuration.

Feature Engineering Strategy The feature engineering process was driven by per-pattern analysis of Fan_Out detection failures. This targeted approach was effective: adding `pair_tx_count` improved the validation AUPRC from 0.7704 to 0.9087 and raised detection rates across all patterns simultaneously. However, the remaining patterns were not directly targeted with dedicated features. A similar approach applied to these patterns might yield comparable improvements and is left for future work.

Feature Engineering Bias The feature engineering process was driven by XGBoost's per-pattern detection failures, meaning the resulting feature set was optimised for XGBoost rather than the DQN. As the SHAP analysis shows, the DQN fails to exploit `pair_tx_count`, the dominant XGBoost feature,

suggesting the two models may benefit from different feature representations. A DQN-driven feature engineering process may have yielded a more suitable feature set for the DQN.

5.3 Limitations

Synthetic dataset. Although SAML-D was developed in consultation with 8 AML specialists and grounded in existing datasets and money laundering literature Oztas et al. (2023), it remains fully synthetic. The existing datasets it builds upon, including AMLSim, are themselves synthetic, meaning no real transaction data underlies the typology design. As a result, the laundering patterns encoded in SAML-D reflect researcher and expert estimations rather than observed criminal behaviour, and the models trained on this data may not generalise to real banking environments.

Dataset representativeness. SAML-D is centred on UK banking transactions Oztas et al. (2023). As the UK operates outside the EU regulatory framework, the dataset may not fully reflect the transaction behaviour and laundering patterns present in Dutch banking environments.

Single dataset. This research evaluates both models on a single dataset. Results may therefore be specific to the characteristics of SAML-D and may not hold across different datasets or banking contexts.

Classification threshold. This research deliberately avoids fixing a classification threshold, as the optimal operating threshold depends on business context and investigator capacity Attard and de Bruijn (2026) Menon and Nguyen (2026). However, the per-pattern detection analysis during feature engineering requires a threshold to determine which transactions are flagged. A threshold of 0.5 was used for this purpose only, as a neutral default to guide feature engineering decisions rather than as an operational recommendation.

SHAP sample size. SHAP values were computed on a sample of 2,500 test transactions due to the DQN's KernelExplainer being computationally expensive. A preliminary test at n=1,000 produced similar results, suggesting the sample is representative.

DQN run-to-run variability. Each DQN iteration was documented from a single run. Given the model's variability, small performance differences between iterations may reflect randomness rather than the effect of the configuration change.

5.4 Reliability, Validity and Generalisability

Reliability Reproducibility was ensured through a fixed random seed of 42 across all XGBoost experiments. The chronological 70/15/15 split prevents data leakage by ensuring the model is never trained on future transactions, reflecting realistic deployment conditions. DQN results are less reliable due to run-to-run variability as discussed in Section 5.3.

Validity Both models were evaluated under identical conditions on the same dataset using the same metric, ensuring internal validity. AUPRC was selected as the primary metric based on industry practice at ING and ABN AMRO Attard and de Bruijn (2026); Menon and Nguyen (2026), ensuring the evaluation is meaningful in a real-world AML context.

Generalisability XGBoost's high AUPRC of 0.9275 depends heavily on the engineered features, particularly `pair_tx_count`. On datasets without similar feature engineering, performance would likely be considerably lower, as demonstrated by the raw feature baseline of 0.0770. The DQN's poor performance should not be interpreted as evidence that RL cannot work for AML in general, but rather that this specific architecture and setup was insufficient for the complexity of SAML-D.

5.5 Ethical and Societal Reflection

Effective AML detection has direct societal benefits by disrupting criminal activity and protecting the financial system. However, the high false positive rate of 61 false alarms per genuine case at XGBoost's operating threshold raises ethical concerns. Legitimate customers incorrectly flagged as suspicious may face delayed transactions or restricted banking access. This underlines the importance of human review in the alert pipeline, as confirmed by both ING and ABN AMRO Attard and de Bruijn (2026); Menon and Nguyen (2026), and the need to set the operating threshold in consultation with business stakeholders rather than optimising recall alone. The DQN's score clustering around 0.5

means that any threshold producing acceptable recall would simultaneously generate an unworkable volume of false alarms, making it unsuitable for production deployment in its current form.

6 Conclusion

This research investigated to what extent can Reinforcement Learning effectively detect money laundering, and how does it compare to leading supervised learning methods on transaction data.

Subquestion 1: How can Reinforcement Learning be applied to AML transaction monitoring? A Deep Q-Network was applied to AML transaction monitoring by framing each transaction as an independent single-step episode within a Markov Decision Process. Each transaction is represented as a 91-element feature vector, and the agent learns to classify transactions as suspicious or normal through a reward function that penalises missed laundering cases more heavily than false positives. This formulation follows the ICMDP framework of Lin et al. Lin et al. (2019), with the reward function parameter λ tested across values proposed by both Lin et al. and Zhinin-Vera et al. Zhinin-Vera et al. (2020). This demonstrates that RL can be applied to tabular AML data without requiring graph structure or sequential transaction modelling.

Subquestion 2: How does Reinforcement Learning perform compared to leading supervised learning methods? XGBoost substantially outperforms the DQN across all evaluated dimensions. The performance gap is fundamental rather than a consequence of insufficient tuning, as the untuned XGBoost baseline already exceeded the best DQN result. For a detailed breakdown of results, refer to Section 4.

Subquestion 3: What are the benefits and limitations of Reinforcement Learning for AML? The primary benefit of the DQN is its ability to continue training incrementally when new data arrives, without requiring a full retrain. This is a meaningful operational advantage in a domain where transaction patterns evolve over time. However, the DQN's limitations in this research are significant: it fails to exploit the most informative features, produces scores that cluster around 0.5 with no meaningful discrimination, and requires considerably more computational resources and configuration effort than XGBoost.

Main Research Question Reinforcement Learning, in the form of a Deep Q-Network with the ICMDP reward structure, cannot effectively detect money laundering at a level comparable to XGBoost on transaction data. XGBoost outperforms the DQN across all evaluated dimensions and is the recommended approach for AML transaction monitoring on tabular data.

References

Alarab, I., Prakoonwit, S., and Nacer, M. I. (2020). Competence of graph convolutional networks for anti-money laundering in bitcoin blockchain. In *Proceedings of the 2020 5th International Conference on Machine Learning Technologies*, page 23–27. Association for Computing Machinery. https://doi.org/10.1145/3409073.3409080.

Attard, M. and de Bruijn, L. (2026). Machine learning binnen ING. Guest lecture at Hogeschool van Amsterdam. Personal notes taken during presentation.

Bryansky, S., Tony, krivoship, CPMP, and Giba (2019). 2nd solution, CPMP view. https://www.kaggle.com/competitions/ieee-fraud-detection/writeups/2-uncles-and-3-puppies-2nd-solution-cpmp-view.

Cox, D. (2012). What is money laundering? In *Handbook of Anti Money Laundering*, chapter 1, pages 5–13. John Wiley Sons, Ltd. https://doi.org/10.1002/9780470685280.ch1.

Cui, Y., Han, X., Chen, J., Zhang, X., Yang, J., and Zhang, X. (2025). FraudGNN-RL: A graph neural network with reinforcement learning for adaptive financial fraud detection. *IEEE Open Journal of the Computer Society*, 6:426–437. https://doi.org/10.1109/OJCS.2025.3543450.

Deotte, C. and Yakovlev, K. (2019). 1st place solution - part 2. https://www.kaggle.com/competitions/ieee-fraud-detection/writeups/fraudsquad-1st-place-solution-part-2.

Dobbelstein, G. and Yeng, L. (2026). Aml at DLL. Personal interview conducted at De Lage Landen, Eindhoven, May 20, 2026.

European Parliament and Council of the European Union (2016). Regulation (EU) 2016/679 — general data protection regulation (GDPR). https://www.privacy-regulation.eu/nl/artikel-9-verwerking-van-bijzondere-categorieen-van-persoonsgegevens-EU-AVG.htm.

Financial Crimes Enforcement Network (FinCEN) (n.d.). What is money laundering? Retrieved March 12, 2026, from https://www.fincen.gov/what-money-laundering.

FIU-the Netherlands (2023). What is money laundering? Retrieved March 12, 2026, from https://www.fiu-nederland.nl/en/home/what-is-money-laundering/.

Han, J., Kamber, M., and Pei, J. (2011). *Data Mining: Concepts and Techniques*. Morgan Kaufmann, 3 edition.

Harris, D. A., Pyndiura, K. L., Sturrock, S. L., and Christensen, R. A. (2021). Using real-world transaction data to identify money laundering: Leveraging traditional regression and machine learning techniques. *STEM Fellowship Journal*, 7(1):1–11. https://doi.org/10.17975/sfj-2021-006.

IEEE Computational Intelligence Society (2019). IEEE-CIS fraud detection: Improve fraud prevention accuracy. Kaggle. https://www.kaggle.com/competitions/ieee-fraud-detection/overview.

Jensen, R. I. T., Ferwerda, J., Jørgensen, K. S., Jensen, E. R., Borg, M., Krogh, M. P., Jensen, J. B., and Iosifidis, A. (2023). A synthetic data set to benchmark anti-money laundering methods. *Scientific Data*, 10. https://doi.org/10.1038/s41597-023-02569-2.

Lin, E., Chen, Q., and Qi, X. (2019). Deep reinforcement learning for imbalanced classification. https://arxiv.org/abs/1901.01379.

Mahootiha, M. (2023). money laundering data. https://www.kaggle.com/datasets/maryam1212/money-laundering-data.

Menon, S. and Nguyen, T. (2026). AML at ABN AMRO. Guest lecture at Hogeschool van Amsterdam. Personal notes taken during presentation.

Ministerie van Financiën (2008). Wet ter voorkoming van witwassen en financieren van terrorisme (Wwft). Retrieved March 14, 2026, from https://wetten.overheid.nl/BWBR0024282/2026-01-01#Hoofdstuk3.

Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., Graves, A., Riedmiller, M., Fidjeland, A. K., Ostrovski, G., Petersen, S., Beattie, C., Sadik, A., Antonoglou, I., King, H., Kumaran, D., Wierstra, D., Legg, S., and Hassabis, D. (2015). Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533. https://doi.org/10.1038/nature14236.

Oztas, B., Cetinkaya, D., Adedoyin, F., Budka, M., Dogan, H., and Aksu, G. (2023). Enhancing anti-money laundering: Development of a synthetic transaction monitoring dataset. In *2023 IEEE International Conference on e-Business Engineering (ICEBE)*, pages 47–54. https://doi.org/10.1109/ICEBE59045.2023.00028.

Probst, P., Boulesteix, A.-L., and Bischl, B. (2019). Tunability: Importance of hyperparameters of machine learning algorithms. *Journal of Machine Learning Research*, 20:1–32. http://jmlr.org/papers/v20/18-444.html.

Qayoom, A., Khuhro, M., Kumar, K., Waqas, M., Saeed, U., ur Rehman, S., Wu, Y., and Wang, S. (2024). A novel approach for credit card fraud transaction detection using deep reinforcement learning scheme. *PeerJ Computer Science*, 10:e1998. https://doi.org/10.7717/peerj-cs.1998.

Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., and Salakhutdinov, R. (2014). Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15:1929–1958. http://jmlr.org/papers/v15/srivastava14a.html.

Team Lions (2019). 5th place solution: Lions. https://www.kaggle.com/competitions/ieee-fraud-detection/writeups/lions-5th-place-solution-lions.

Tertychnyi, P., Godgildieva, M., Dumas, M., and Ollikainen, M. (2022). Time-aware and interpretable predictive monitoring system for anti-money laundering. *Machine Learning with Applications*, 8:100306. https://doi.org/10.1016/j.mlwa.2022.100306.

Vimal, S., Kayathwal, K., Wadhwa, H., and Dhama, G. (2021). Application of deep reinforcement learning to payment fraud. https://arxiv.org/abs/2112.04236.

Weber, M., Chen, J., Suzumura, T., Pareja, A., Ma, T., Kanezashi, H., Kaler, T., Leiserson, C. E., and Schardl, T. B. (2018). Scalable graph learning for anti-money laundering: A first look. https://arxiv.org/abs/1812.00076.

Weber, M., Domeniconi, G., Chen, J., Weidele, D. K. I., Bellei, C., Robinson, T., and Leiserson, C. E. (2019). Anti-money laundering in bitcoin: Experimenting with graph convolutional networks for financial forensics. https://arxiv.org/abs/1908.02591.

XGBoost Developers (2024). XGBoost parameters. Retrieved May 2026, from https://xgboost.readthedocs.io/en/stable/parameter.html.

Youssef, B., Bouchra, F., and Brahim, O. (2023). State of the art literature on anti-money laundering using machine learning and deep learning techniques. In *The 3rd International Conference on Artificial Intelligence and Computer Vision (AICV2023), March 5–7, 2023*, pages 77–90. Springer Nature Switzerland. https://rdcu.be/e8mZQ.

Zhinin-Vera, L., Chang, O., Valencia-Ramos, R., and Velastegui, R. (2020). Q-credit card fraud detector for imbalanced classification using reinforcement learning. In *Proceedings of the 12th International Conference on Agents and Artificial Intelligence (ICAART)*. SCITEPRESS. https://www.scitepress.org/Papers/2020/91561/91561.pdf.

A Requirements

This chapter defines the legal, functional, and technical requirements that shape the design and evaluation of the proposed model.

A.1 Legal & Ethical Requirements

This research operates within the Dutch legal framework for financial crime detection and data privacy. This highlights two particular laws: *Wet ter voorkoming van witwassen en financieren van terrorisme (Wwft)*, the Dutch implementation of the EU Anti-Money Laundering Directive Ministerie van Financiën (2008) and *General Data Protection Regulation (GDPR)*, known locally as Algemene Verordening Gegevensbescherming (AVG) European Parliament and Council of the European Union (2016).

Table 17: Legal & Ethical Requirements

ID	Requirement	Source
LERQ1	When a transaction is flagged as suspicious, the model must be able to explain why.	Wwft, Art. 16.2e Ministerie van Financiën (2008)
LERQ2	The use of the following personal attributes is explicitly prohibited in model training: race, ethnic background, political opinion, religious or philosophical beliefs, trade union membership, genetic data, biometric data, health data, and data concerning a natural person's sexual behaviour or sexual orientation.	GDPR, Art. 9.1 European Parliament and Council of the European Union (2016)

A.2 Functional Requirements

The following functional requirements define what the model must be capable of doing:

Table 18: Functional Requirements

ID	Requirement	Source
FR2	The model must output a continuous suspiciousness score per transaction.	Required for AUPRC, Industry practice Attard and de Bruijn (2026)
FR3	The model must provide a feature-level explanation for each flagged transaction using SHAP values.	Wwft, Art. 16.2e Ministerie van Financiën (2008);
FR4	The model must be evaluated using AUPRC as the primary metric.	Industry practice Attard and de Bruijn (2026); Menon and Nguyen (2026)
FR5	Both the RL model and the XGBoost baseline must be trained and evaluated on the same dataset under the same conditions to ensure a fair comparison.	Research scope

A.3 Technical Requirements

The following technical requirements define how and in what circumstances the model must be built:

Table 19: Technical Requirements

ID	Requirement	Source
TR1	The dataset must be tabular	Research scope
TR2	The dataset must be anonymized, containing no personal identifiers.	GDPR, Art. 9.1 European Parliament and Council of the European Union (2016)
TR3	The dataset must not contain cryptocurrency transactions.	Research Scope
TR4	A Deep Q-Network architecture must be used for the RL-model	Related Work Cui et al. (2025); Qayoom et al. (2024)
TR5	The baseline model must be implemented using XGBoost.	Literature Youssef et al. (2023) and Industry practice Attard and de Bruijn (2026); Menon and Nguyen (2026)

B SAML-D Feature Overview

Table 20: SAML-D Feature Overview

Category	Feature	Description	Example
Transaction Features	Time	Time of transaction	09:32:11
	Date	Date of transaction	2022-03-14
	Sender_account	Sender bank account ID	7850710817
	Receiver_account	Receiver bank account ID	9747648665
	Amount	Transaction amount (decimal)	5058.98
	Payment_type	Payment type	Cash Deposit
	Payment_currency	Sender account currency	UK Pounds
	Received_currency	Receiver account currency	UK Pounds
	Sender_bank_location	Sender bank location	UK
	Receiver_bank_location	Receiver bank location	UK
Label Features	Is_laundering	Binary laundering indicator	0, 1
	Laundering_type	Type of laundering pattern	Fan_Out

Table 21: SAML-D Categorical Feature Values

Feature	Possible Values
Payment_type	ACH, Cash Deposit, Cash Withdrawal, Cheque, Credit card, Cross-border, Debit card
Payment_currency	Albanian lek, Dirham, Euro, Indian rupee, Mexican Peso, Moroccan dirham, Naira, Pakistani rupee, Swiss franc, Turkish lira, UK pounds, US dollar, Yen
Received_currency	Albanian lek, Dirham, Euro, Indian rupee, Mexican Peso, Moroccan dirham, Naira, Pakistani rupee, Swiss franc, Turkish lira, UK pounds, US dollar, Yen
Sender_bank_location	Albania, Austria, France, Germany, India, Italy, Japan, Mexico, Morocco, Netherlands, Nigeria, Pakistan, Spain, Switzerland, Turkey, UAE, UK, USA
Receiver_bank_location	Albania, Austria, France, Germany, India, Italy, Japan, Mexico, Morocco, Netherlands, Nigeria, Pakistan, Spain, Switzerland, Turkey, UAE, UK, USA
Laundering_type	Behavioural_Change_1, Behavioural_Change_2, Bipartite, Cash_Withdrawal, Cycle, Deposit-Send, Fan_In, Fan_Out, Gather-Scatter, Layered_Fan_In, Layered_Fan_Out, Normal_Cash_Deposits, Normal_Cash_Withdrawal, Normal_Fan_In, Normal_Fan_Out, Normal_Foward, Normal_Group, Normal_Mutual, Normal_Periodical, Normal_Plus_Mutual, Normal_Small_Fan_Out, Normal_single_large, Over-Invoicing, Scatter-Gather, Single_large, Smurfing, Stacked_Bipartite, Structuring

Table 22: Payment Type Descriptions

Payment Type	Description
ACH	Automated Clearing House transfer — an electronic bank-to-bank transfer commonly used for payroll and bill payments
Cash Deposit	Physical cash deposited directly into a bank account
Cash Withdrawal	Physical cash withdrawn from a bank account
Cheque	Payment made via a written paper order instructing a bank to pay a specified amount
Credit card	Payment made using a credit card
Cross-border	An international transfer sent across country borders
Debit card	Payment made using a debit card, directly debiting the sender's account

C Laundering Type Descriptions

Table 23: SAML-D Laundering Typology Descriptions

Typology	**Class**	**Description**
Normal_Cash_Deposits	Normal	Regular cash deposits consistent with expected customer behaviour
Normal_Cash_Withdrawal	Normal	Regular cash withdrawals consistent with expected customer behaviour
Normal_Fan_In	Normal	Multiple senders paying one receiver, consistent with legitimate business activity
Normal_Fan_Out	Normal	One sender paying multiple receivers, consistent with legitimate business activity
Normal_Foward	Normal	An account receives a transfer and immediately forwards it to another account, acting as a pass-through.
Normal_Group	Normal	Group of accounts transacting among themselves in a normal pattern
Normal_Mutual	Normal	Two accounts transacting back and forth in a normal pattern
Normal_Periodical	Normal	Regular periodic payments consistent with subscriptions or salary
Normal_Plus_Mutual	Normal	Combination of normal forward and mutual transactions
Normal_Small_Fan_Out	Normal	One sender making a small number of payments to different receivers
Normal_single_large	Normal	A single large transaction consistent with legitimate activity
Fan_In	Suspicious	Multiple accounts funnelling money into a single account to aggregate funds
Fan_Out	Suspicious	A single account rapidly distributing funds to many receivers to obscure origin
Layered_Fan_In	Suspicious	Multi-hop version of Fan_In where funds pass through intermediate accounts before aggregating
Layered_Fan_Out	Suspicious	Multi-hop version of Fan_Out where funds pass through intermediate accounts before dispersing
Bipartite	Suspicious	Two groups of accounts transacting between each other to obscure the flow of funds
Stacked_Bipartite	Suspicious	Multiple layers of bipartite transactions stacked to further obscure the origin
Scatter-Gather	Suspicious	Funds are first scattered across many accounts and then gathered back into one
Gather-Scatter	Suspicious	Funds are first gathered from many accounts and then scattered to obscure the trail
Cycle	Suspicious	Money moves in a circular chain of accounts eventually returning to the origin
Smurfing	Suspicious	Large amounts are broken into many smaller transactions to avoid detection thresholds
Structuring	Suspicious	Transactions are deliberately kept just below reporting thresholds
Single_large	Suspicious	A single unusually large transaction inconsistent with the account's normal behaviour
Behavioural_Change_1	Suspicious	An account that deviates from its Normal_Group pattern by suddenly transacting with new unknown accounts
Behavioural_Change_2	Suspicious	An account that deviates from its Normal_Group pattern by suddenly transacting with new accounts in high-risk locations
Over-Invoicing	Suspicious	Inflated invoice amounts used to move money between accounts under the guise of legitimate trade
Deposit-Send	Suspicious	Cash is deposited and immediately sent onwards to obscure the origin
Cash_Withdrawal	Suspicious	Suspicious cash withdrawal pattern inconsistent with normal account behaviour

D Feature Engineering Iterations

This appendix provides the full iteration-by-iteration history of the feature engineering process referenced in Section 3.3.3. Each iteration was driven by per-pattern detection failures identified through analysis of the XGBoost model's validation results, as described in the main text.

D.1 Feature Engineering — Iteration 1: Raw Features

Before engineering any features, a preliminary XGBoost model was trained on the 12 raw SAML-D features to identify which laundering patterns the raw features alone could not capture. The raw feature model achieved a validation AUPRC of 0.0770 (Figure 40). Per-pattern analysis revealed that `Fan_Out` was the least detected pattern, with a catch rate of only 18.39% (Figure 41). Examining the raw features of suspicious and normal Fan_Out transactions showed they were nearly indistinguishable (Table 24), directly motivating the first round of feature engineering. Per-pattern detection rates throughout this section are computed using a classification threshold of 0.5 for diagnostic purposes. The final operational threshold is not fixed in this research and is left to business stakeholders, consistent with industry practice Menon and Nguyen (2026).

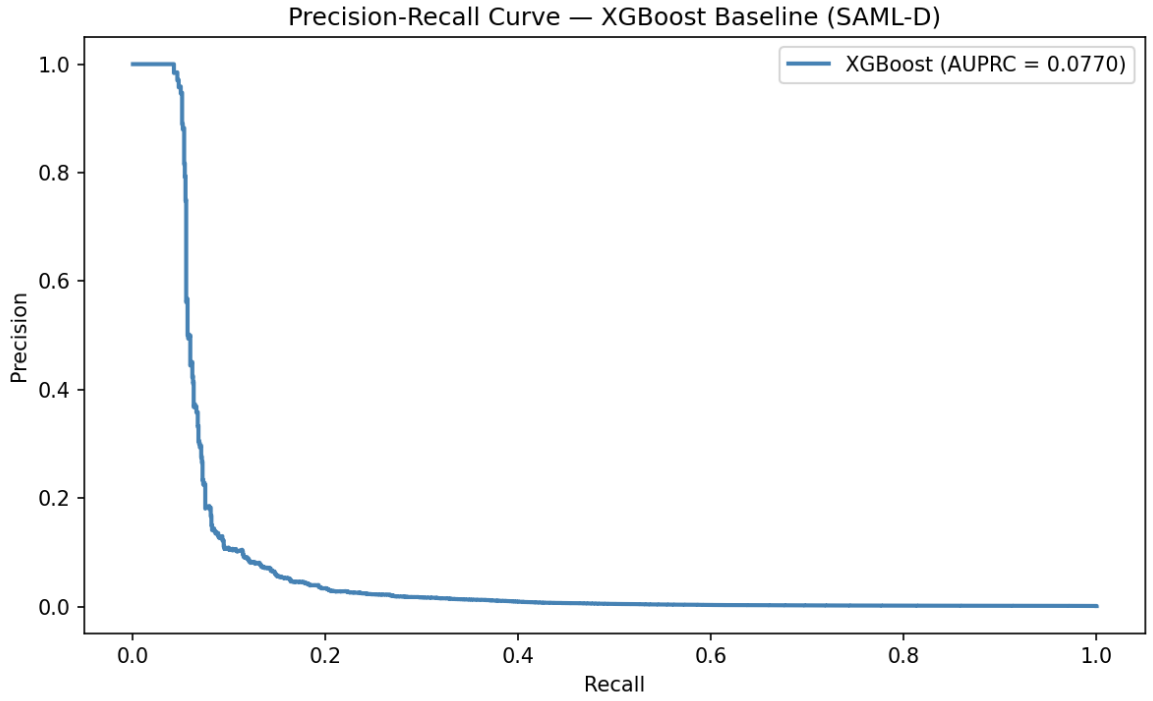

Figure 40: Precision-recall curve on the validation set, Iteration 1: raw features only, Val AUPRC = 0.0770

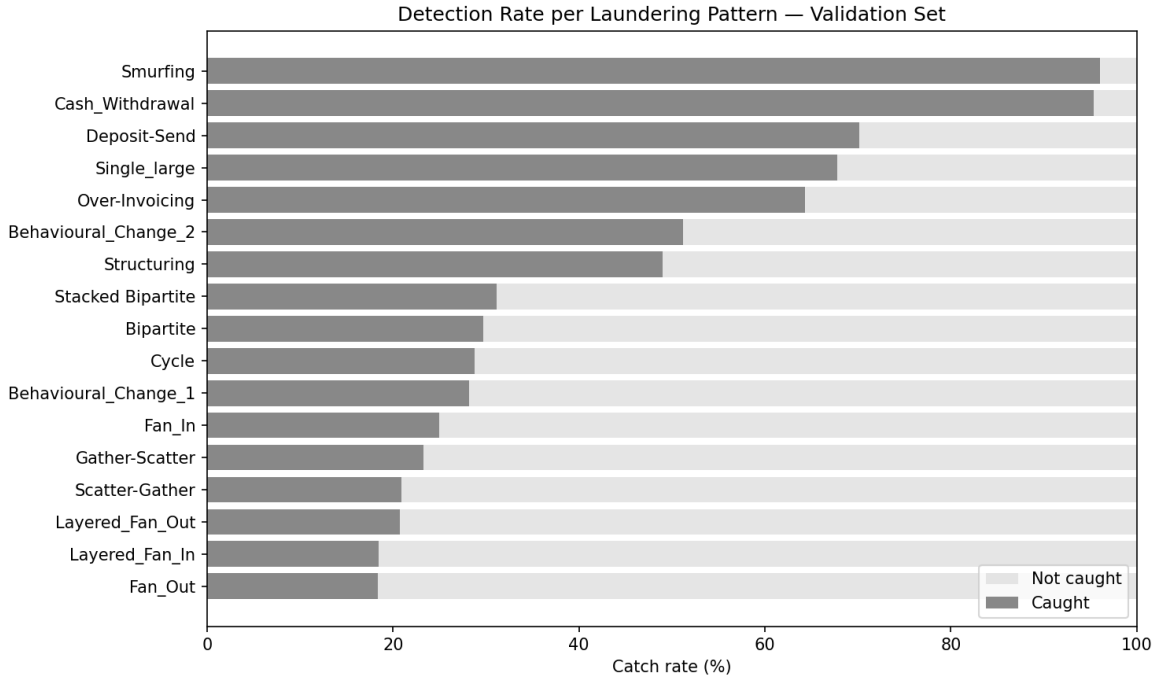

Figure 41: Per-pattern detection rate on the validation set, Iteration 1: raw features only, Val AUPRC = 0.0770. Detection determined at a classification threshold of 0.5.

Table 24: Raw features comparison: Normal_Fan_Out vs Fan_Out (5 examples each)

Class	Time	Date	Sender	Receiver	Amount	Pay. Curr.	Rec. Curr.	Sender Loc.	Rec. Loc.	Pay. Type	Label	Type
Normal	10:35:20	2022-10-07	1491989064	8401255335	6019.64	UK pounds	Dirham	UK	UAE	Cross-border	0	Normal_Fan_Out
Normal	10:35:26	2022-10-07	6116657264	656192169	4738.45	UK pounds	UK pounds	UK	UK	Cheque	0	Normal_Fan_Out
Normal	10:35:29	2022-10-07	7421451752	2755709071	5883.87	Indian rupee	UK pounds	UK	UK	Credit card	0	Normal_Fan_Out
Normal	10:35:46	2022-10-07	3709430533	9172843471	5274.76	UK pounds	UK pounds	UK	UK	Debit card	0	Normal_Fan_Out
Normal	10:36:13	2022-10-07	8389148416	721717864	16974.63	UK pounds	UK pounds	UK	UK	Credit card	0	Normal_Fan_Out
Suspicious	07:58:09	2022-10-15	6652024436	9685947642	5936.38	UK pounds	UK pounds	UK	UK	ACH	1	Fan_Out
Suspicious	23:50:07	2022-10-16	6652024436	2585134008	14610.64	UK pounds	Moroccan dirham	UK	Morocco	Cross-border	1	Fan_Out
Suspicious	04:57:10	2022-10-17	6652024436	9604482457	5894.17	Naira	UK pounds	UK	UK	Debit card	1	Fan_Out
Suspicious	17:21:59	2022-10-17	6652024436	5601106416	7464.02	UK pounds	UK pounds	UK	UK	ACH	1	Fan_Out
Suspicious	20:07:24	2022-10-18	6652024436	9252458648	8632.48	UK pounds	UK pounds	UK	UK	ACH	1	Fan_Out

D.2 Feature Engineering — Iteration 2: Account-level Aggregates

To give the model context about the accounts behind each transaction, statistical profiles were constructed from the training set for each sender and receiver account, following the approach of Harris et al. Harris et al. (2021), who demonstrated that adding account-level aggregates improves money laundering detection performance.

These profiles captured features such as transaction count, average amount, standard deviation, minimum and maximum amounts, number of unique counterparties, unique currencies, and unique locations. Fan ratio features (`fan_out_ratio`, `fan_in_ratio`) and cross-border flags (`is_cross_currency`, `is_cross_border`) were also derived. Profiles were joined onto the validation and test sets by account ID. Accounts not seen during training received a value of 0 for all profile features, reflecting the assumption that no prior history is known. The `Laundering_type` column was excluded from the feature set as it would constitute label leakage, though it was retained in the saved splits for post-hoc per-pattern analysis. A full overview of all engineered features is provided in Appendix D.4.

This expanded the feature set from 12 to 26 features and improved the validation AUPRC from 0.0770 to 0.7704 (Figure 42). Almost all patterns were detected at a higher rate than in the raw model (Figure 43). However, `Fan_Out` remained on the lower end of the spectrum (Figure 44). Analysis of missed Fan_Out cases (Table 25) revealed a consistent pattern: nearly all missed transactions had `receiver_unique_senders` of 0 or 1, meaning the receiver had little or no transaction history in the

training set. These receivers were indistinguishable from normal accounts, motivating a third round of feature engineering.

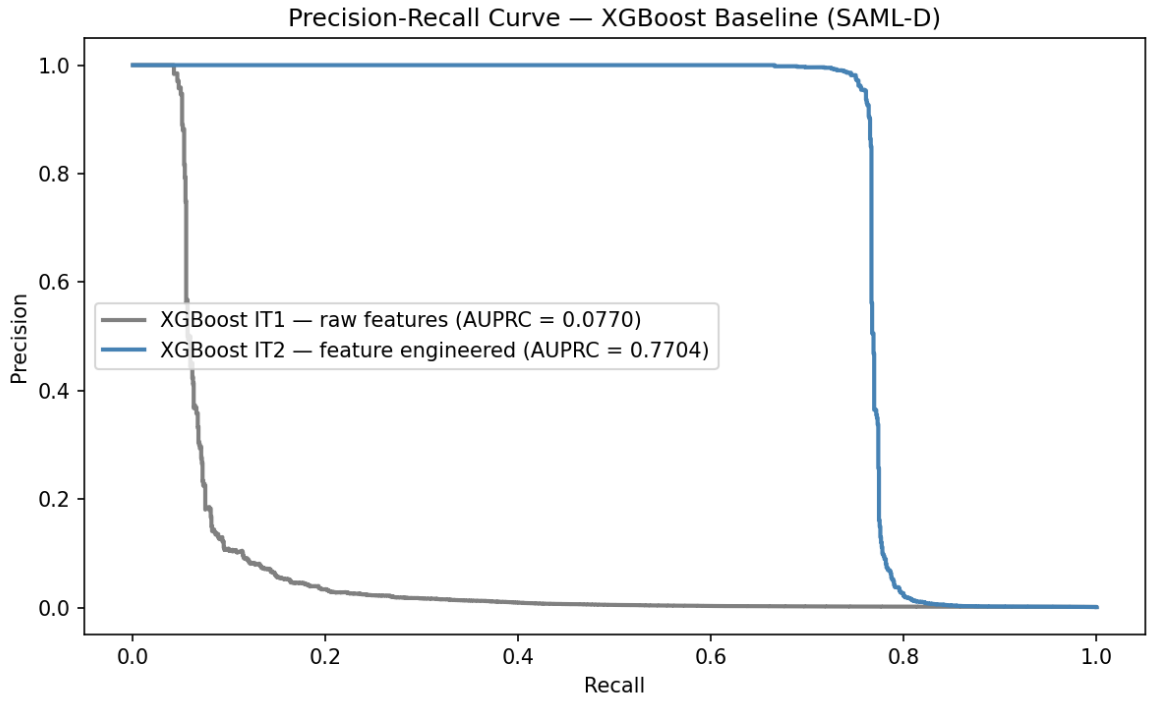

Figure 42: Precision-recall curve on the validation set, Iteration 2: account-level aggregates, Val AUPRC = 0.7704

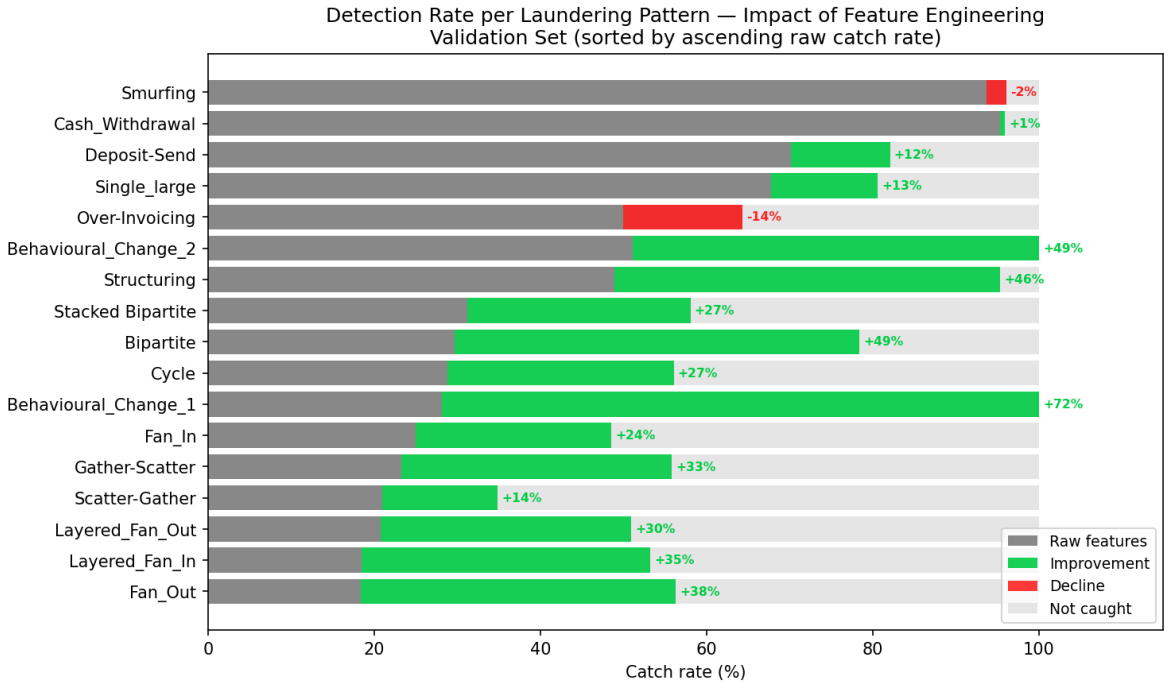

Figure 43: Per-pattern detection rate change from Iteration 1 to Iteration 2 on the validation set, sorted by ascending raw catch rate. Grey bars show the raw baseline, green indicates improvement and red indicates decline. Detection determined at a classification threshold of 0.5.

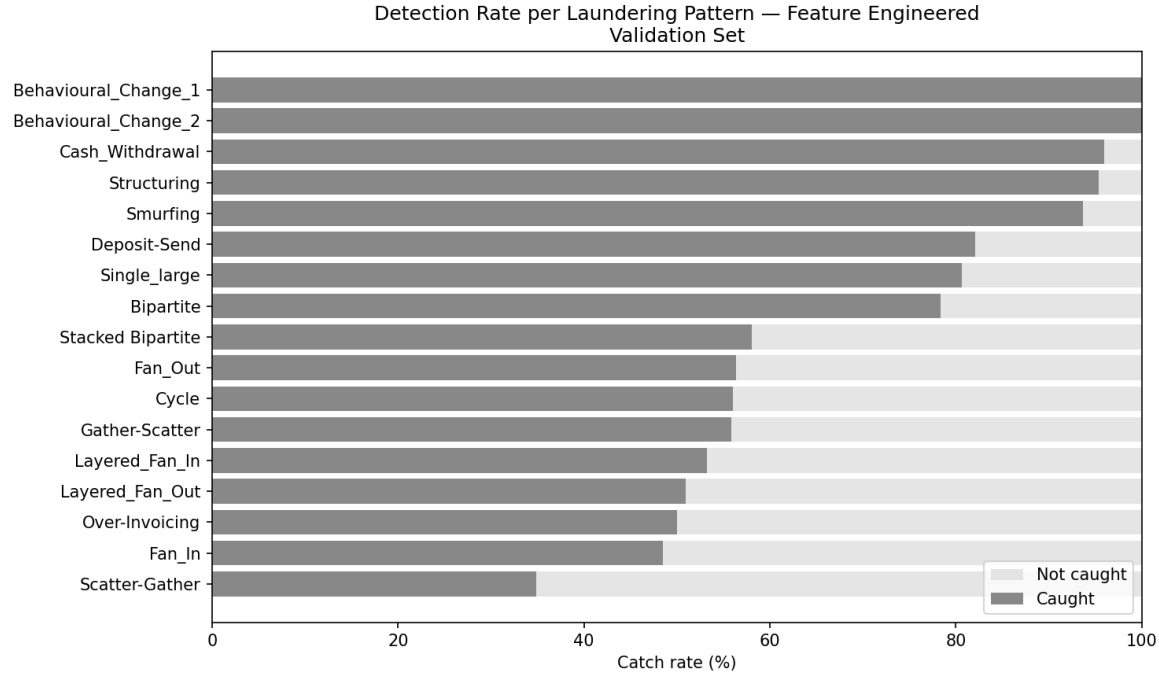

Figure 44: Per-pattern detection rate after feature engineering on the validation set, Iteration 2: account-level aggregates, Val AUPRC = 0.7704. Detection determined at a classification threshold of 0.5.

Table 25: Fan_Out analysis: caught vs missed vs normal

Sender	Receiver	Label	Type	sen_tx_n	sen_uniq_rec	rec_tx_n	rec_uniq_sen
Suspicious — correctly caught							
9163385117	4418736003	1	Fan_Out	160	23	107	10
8149005840	7507462899	1	Fan_Out	7	3	4	2
5594972698	4656627785	1	Fan_Out	9	3	8	2
9163385117	3462329312	1	Fan_Out	160	23	7	2
5594972698	3038316768	1	Fan_Out	9	3	11	2
Suspicious — missed							
9163385117	9023181723	1	Fan_Out	160	23	2	1
8149005840	8043005405	1	Fan_Out	7	3	3	1
5594972698	7597465145	1	Fan_Out	9	3	6	1
5594972698	8694554099	1	Fan_Out	9	3	11	1
4067269238	8622050490	1	Fan_Out	2	2	7	1
Normal — correctly ignored							
4300217235	963495531	0	Normal_Fan_Out	383	48	8	1
4752527973	2150770991	0	Normal_Fan_Out	295	37	11	1
6782294771	1934717223	0	Normal_Fan_Out	353	48	7	1
205877386	6531558211	0	Normal_Fan_Out	348	36	10	1
432518499	3053622551	0	Normal_Fan_Out	465	54	9	1
Normal — false positive							
5997554145	4699555198	0	Normal_Fan_Out	4	3	0	0
5997554145	4699555198	0	Normal_Fan_Out	4	3	0	0
2235590128	4408629504	0	Normal_Fan_Out	52	15	0	0
5288254056	6265384966	0	Normal_Fan_Out	16	5	0	0
5288254056	4891437151	0	Normal_Fan_Out	16	5	0	0

D.3 Feature Engineering — Iteration 3: Pair History

To address the remaining Fan_Out detection gap, a new feature `pair_tx_count` was introduced. Analysis of missed Fan_Out cases in Iteration 2 revealed that suspicious transactions were concentrated in new sender-receiver relationships with little prior transaction history, which the existing account-level features could not capture. `pair_tx_count` encodes this directly, representing the number of times a specific sender-receiver pair had transacted together in the training set. This brought the total feature count to 27.

Adding `pair_tx_count` improved the validation AUPRC from 0.7704 to 0.9087 (Figure 45). All per-pattern detection rates were higher or equal compared to the previous iteration (Figure 46). The targeted pattern `Fan_Out` improved by 31%, and comparable patterns such as `Layered_Fan_Out`, `Fan_In`, and `Layered_Fan_In` improved significantly by 38%, 35%, and 19% respectively.

The patterns with the lowest detection rates are `Scatter-Gather` (67.44%), `Layered_Fan_In` (71.74%), and `Cycle` (80.30%) (Figure 47). What these patterns have in common is that they involve money moving across multiple accounts over time. Capturing this sequential movement would require rolling time window features or graph-based approaches, which fall outside the scope of this research.

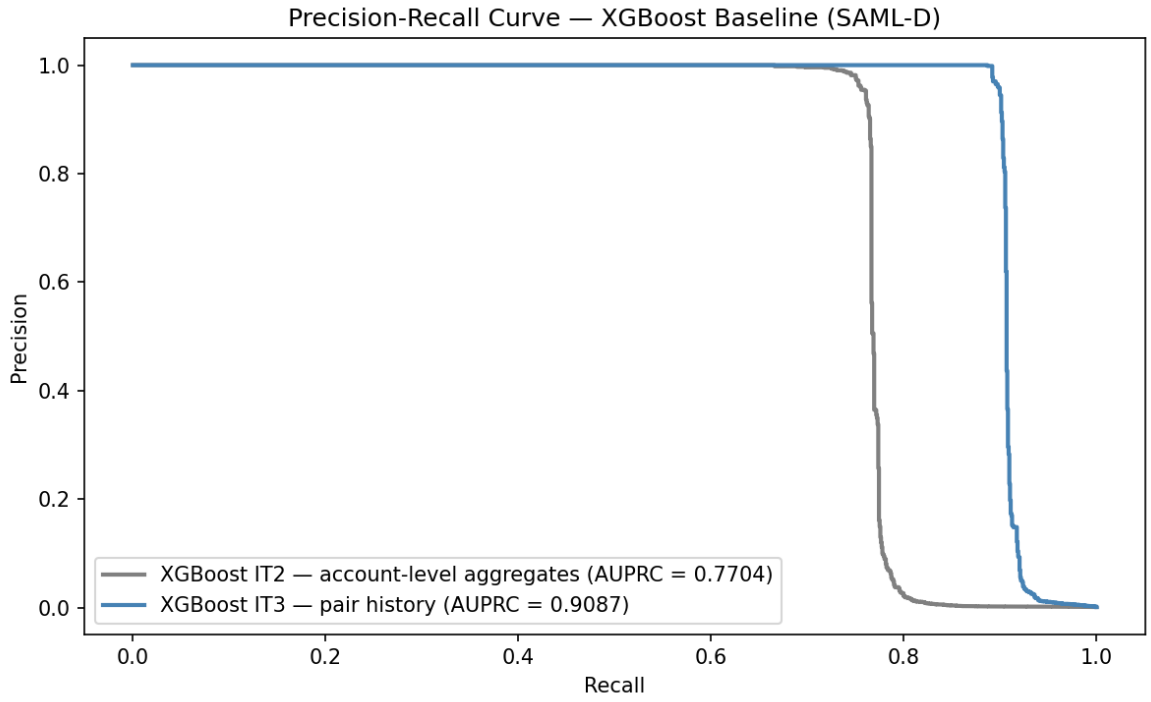

Figure 45: Precision-recall curve on the validation set, Iteration 3: pair history added, Val AUPRC = 0.9087

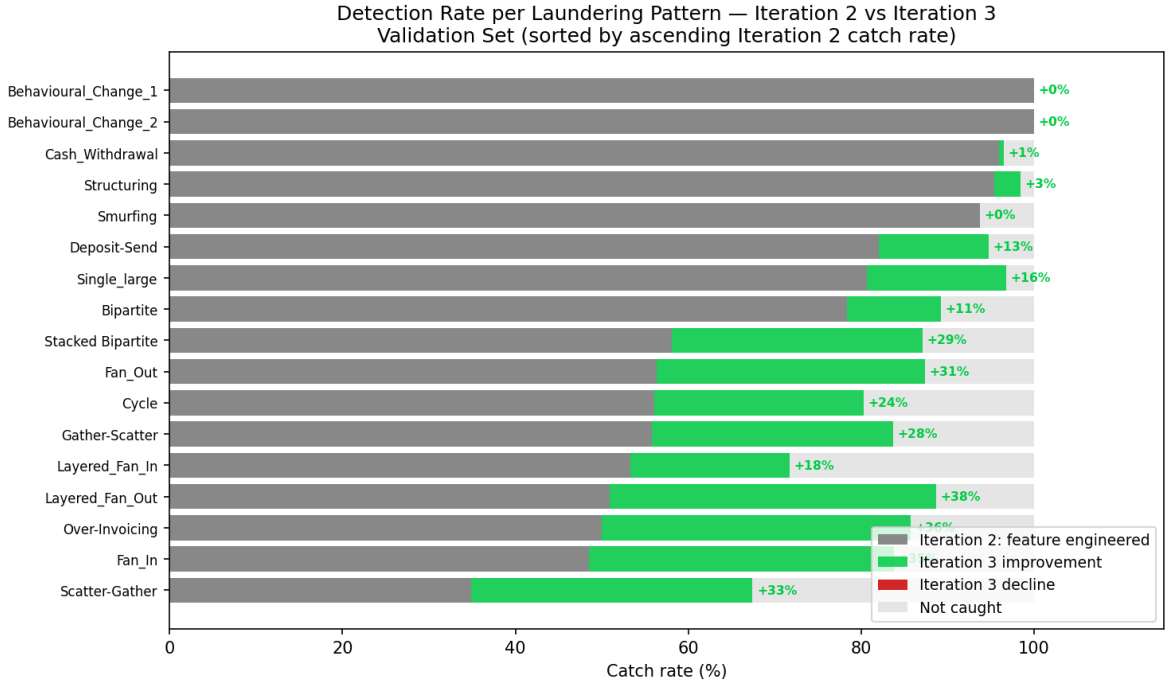

Figure 46: Per-pattern detection rate change from Iteration 2 to Iteration 3 on the validation set, sorted by ascending Iteration 2 catch rate. Grey bars show the Iteration 2 baseline, green indicates improvement and red indicates decline. Detection determined at a classification threshold of 0.5.

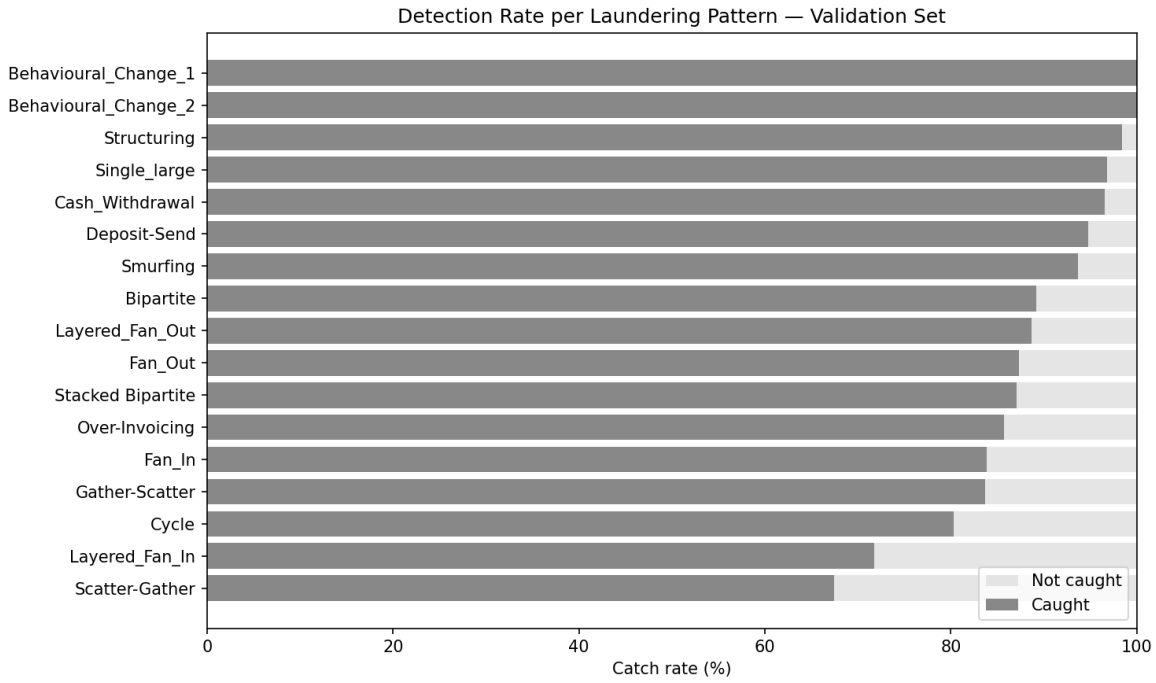

Figure 47: Per-pattern detection rate on the validation set, Iteration 3: pair history added, Val AUPRC = 0.9087, sorted by ascending catch rate. Detection determined at a classification threshold of 0.5.

D.4 Engineered Feature Overview

Table 26 summarises all features added across the iterations described above.

Table 26: Engineered feature overview

Feature	Description
sender_tx_count	Total number of transactions sent by this account in the training set
sender_amount_mean	Mean transaction amount sent by this account
sender_amount_std	Standard deviation of transaction amounts sent by this account
sender_amount_min	Minimum transaction amount sent by this account
sender_amount_max	Maximum transaction amount sent by this account
sender_unique_receivers	Number of unique receiver accounts this sender has sent to
sender_unique_currencies	Number of unique payment currencies used by this sender
sender_unique_locations	Number of unique sender bank locations associated with this account
receiver_tx_count	Total number of transactions received by this account in the training set
receiver_amount_mean	Mean transaction amount received by this account
receiver_amount_std	Standard deviation of transaction amounts received by this account
receiver_unique_senders	Number of unique sender accounts that have sent to this receiver
receiver_unique_currencies	Number of unique received currencies for this account
receiver_unique_locations	Number of unique receiver bank locations associated with this account
fan_out_ratio	Ratio of unique receivers to total transactions sent by this sender
fan_in_ratio	Ratio of unique senders to total transactions received by this receiver
is_cross_currency	Binary flag indicating whether the payment and received currencies differ
is_cross_border	Binary flag indicating whether the sender and receiver bank locations differ
pair_tx_count	Number of times this specific sender-receiver pair has transacted together in the training set

E XGBoost Iteration Details

Iteration 1: Default hyperparameters Iteration 1 trained XGBoost using default hyperparameters, with only the parameters strictly required by the dataset and evaluation methodology overridden. `scale_pos_weight` was set to ∼982, the ratio of normal to laundering transactions in the training set, to handle the extreme class imbalance as recommended by the XGBoost documentation XGBoost Developers (2024). `eval_metric` was set to `aucpr` to match the primary evaluation metric, `early_stopping_rounds` to 50 to prevent overfitting, `enable_categorical` to `True` for native categorical support, and `random_state` to 42 for reproducibility. All other parameters remained at XGBoost defaults. This iteration achieved a validation AUPRC of 0.9087.

Table 27: XGBoost Iteration 1 configuration

Parameter	Value	Justification
`scale_pos_weight`	∼982	Handles class imbalance, recommended by XGBoost documentation XGBoost Developers (2024)
`eval_metric`	aucpr	Primary evaluation metric
`early_stopping_rounds`	50	Prevents overfitting
`enable_categorical`	True	Native categorical support
`random_state`	42	Reproducibility
`remaining parameters`	XGBoost defaults	Recommended configuration XGBoost Developers (2024)

Iteration 2: Grid Search In an attempt to improve upon the baseline, Iteration 2 applied a grid search as a structured first step before continuous optimisation, covering the parameters identified as most tunable by Probst et al. Probst et al. (2019), who benchmarked XGBoost hyperparameter sensitivity across 38 datasets and identified `eta` and `booster` as the two most impactful parameters. The search was expanded to also include `subsample` and `min_child_weight`, which rank among the next most tunable parameters according to Probst et al. Parameter values were derived from three sources: the 5th and 95th percentile optimal values reported by Probst et al., competition-validated values from the 1st place IEEE-CIS Fraud Detection solution Deotte and Yakovlev (2019), and XGBoost documentation defaults XGBoost Developers (2024). All fixed parameters from Iteration 1 were retained. The grid search evaluated all 150 combinations on the validation set, with the test set remaining untouched throughout. This iteration achieved a validation AUPRC of 0.9107.

Table 28: XGBoost Iteration 2 grid search parameter ranges

Parameter	Values Tested	Source
`eta`	[0.002, 0.02, 0.18, 0.3, 0.355]	Probst et al. q0.05; Deotte & Yakovlev; midpoint; default; Probst et al. q0.95
`subsample`	[0.5, 0.75, 0.8, 0.958, 1.0]	Probst et al. q0.05; midpoint; Deotte & Yakovlev; Probst et al. q0.95; default
`min_child_weight`	[1, 4, 7]	Probst et al. q0.05 & default; midpoint; q0.95
`booster`	[gbtree, dart]	All boosters supporting categorical features

Iteration 3: Bayesian Optimisation Iteration 2 explored a discrete set of parameter values. To search the same parameter space more efficiently, Iteration 3 applied Bayesian optimisation using Optuna, which uses past trial results to intelligently select the next parameter combination to evaluate rather than exhaustively testing all combinations.

100 trials were executed, with Trial 22 producing the best result. This iteration achieved a validation AUPRC of 0.9136, establishing the final XGBoost configuration used as the benchmark in this research.

Table 29: XGBoost Iteration 3 Bayesian optimisation search space

Parameter	Search Type	Range	Source
`eta`	Continuous	[0.0001, 1.0]	Full valid range XGBoost Developers (2024)
`subsample`	Continuous	[0.0001, 1.0]	Full valid range XGBoost Developers (2024)
`min_child_weight`	Integer	[1, 7]	Probst et al. q0.05 to q0.95 Probst et al. (2019)
`booster`	Categorical	[gbtree, dart]	Only boosters supporting categorical features

As shown in Table 30 and Figure 48, the improvement from Iteration 1 to Iteration 2 was marginal at 0.0020 validation AUPRC, and from Iteration 2 to Iteration 3 a further 0.0029. The precision-recall curves in Figure 49 confirm this pattern on the test set, with all three iterations producing nearly identical curves. This suggests diminishing returns from continued hyperparameter tuning alone, and further gains would likely require additional feature engineering rather than continued tuning. The Iteration 3 model is therefore used as the XGBoost benchmark for comparison against the Deep Q-Network.

Table 30: XGBoost iteration results

Iteration	Parameters	Val AUPRC	Test AUPRC
Iteration 1	Defaults	0.9087	0.9232
Iteration 2	Grid search	0.9107	0.9260
Iteration 3	Bayesian optimisation	0.9136	0.9275

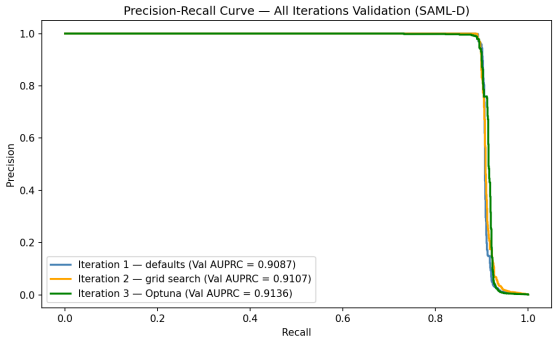

Figure 48: Precision-recall curves for all XGBoost iterations on the validation set

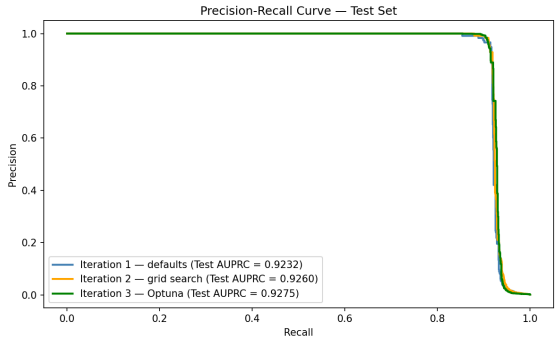

Figure 49: Precision-recall curves for all XGBoost iterations on the test set